
CSE 312

Operating Systems

Spring 2026

IO

© 2013-2026 Yakup Genç

I/O and Operating Systems

- All I/O must go through the operating system
- A typical system has (or could have) many different types of devices
- What kind of software talks to these devices?
- What is the interface between the OS and that software?
- What is the interface to application programs?
- Are there any hardware constraints?

I/O Devices

Some typical device, network,
and bus data rates

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Scanner	400 KB/sec
Digital camcorder	3.5 MB/sec
802.11g Wireless	6.75 MB/sec
52x CD-ROM	7.8 MB/sec
Fast Ethernet	12.5 MB/sec
Compact flash card	40 MB/sec
FireWire (IEEE 1394)	50 MB/sec
USB 2.0	60 MB/sec
SONET OC-12 network	78 MB/sec
SCSI Ultra 2 disk	80 MB/sec
Gigabit Ethernet	125 MB/sec
SATA disk drive	300 MB/sec
Ultrium tape	320 MB/sec
PCI bus	528 MB/sec

I/O Models

- All operating systems impose tight controls on disk access, because the OS itself needs to access the disk and control file storage
- Operating systems frequently provide high-level access to common types of devices: serial ports, printers, tapes, etc.
- Some operating systems permit user-specified complex I/O to some devices (tapes, displays)
- The more “normal” an I/O device is, the more likely it is to be accessed by a standard model
- Try for an I/O model that lets most applications be device-independent

The Unix Model

- Two categories, block devices and character devices
- File systems live on top of block devices
- Block devices are designed for “give me N blocks starting at block number M”
- Almost never used directly by the application programmer
- Character devices — everything else — return a series of unstructured bytes
- Provides a high degree of device independence — for most applications. . .

The Windows Model

- More oriented towards bit maps
- Screens are bitmaps
- Printers are bitmaps
- Disks? Disks use the same block model as in Unix

Networking

- Network devices are completely different
- They're neither block nor character-oriented
- They're used very differently by the OS

I/O Model

- Character devices
- Block devices
- Bitmaps
- Networks

Back to Unix

- All devices have filenames:

```
$ ls --time-style "+" -Ggl \  
/dev/sda1 /dev/pts/6  
crw--w---- 1 136, 6 /dev/pts/6  
brw-rw---- 1 8, 1 /dev/sda1
```

- The “c” indicates a character I/O devices; the “b” indicates a block device
- The “136” and the “8” are major device numbers
- The “6” and the “1” are minor device numbers
- Device files are created with mknod

Major Device Numbers

- Major device numbers indicate the device type. More specifically, they indicate the device driver to be used
- Device driver numbering is very system-specific
- Block and character device numbers are separate namespaces

Minor Device Numbers

- Minor device numbers indicate the unit: which disk drive, which serial port, etc.
- In some cases, the minor device number has other flags encoded in the number
- Example: on Linux, the minor device for a SCSI disk is $16 \times \text{drivenum} + \text{partition}$

Device-Specific Semantics

- Not all devices are alike
- Some provision has to be made for device-specific semantics
- Example: skip a “file” on a tape
- Example: handle special characters from terminals: backspace, ^C, etc.
- Certain operations are handled by ioctl calls
- Tty processing is handled by a “line discipline” that is invoked by all tty-like devices

Device Driver Interface

- All devices have certain standard entry points: open, close, read, write, ioctl, etc.
- Devices implement Unix semantics as best they can
- Device-specific restrictions may, of course, be honored

Block Devices

- Also have a standardized interface, but it's quite different
- Again, primary function is read/write blocks on disk
- Integrated with buffer cache and memory subsystem
- Accessed primarily through the file system
- (Exception: some high-end database systems)
- Still need device-specific commands (see `hdparm` command on Linux)

Disk Scheduling

- Disk seeks are slow
- Rotational delay takes time, too
- Disk I/O driver must optimize throughput

Conflicting Demands

- Fairness: do I/O operations in order of arrival
- Efficiency: don't waste a lot of time seeking
- Promptness: don't delay some requests indefinitely

Two Not So Good Answers

- First come, first served — very inefficient
- Shortest seek first — unfair to requests at the ends of the disk; vulnerable to indefinite overtaking

Elevator Algorithm

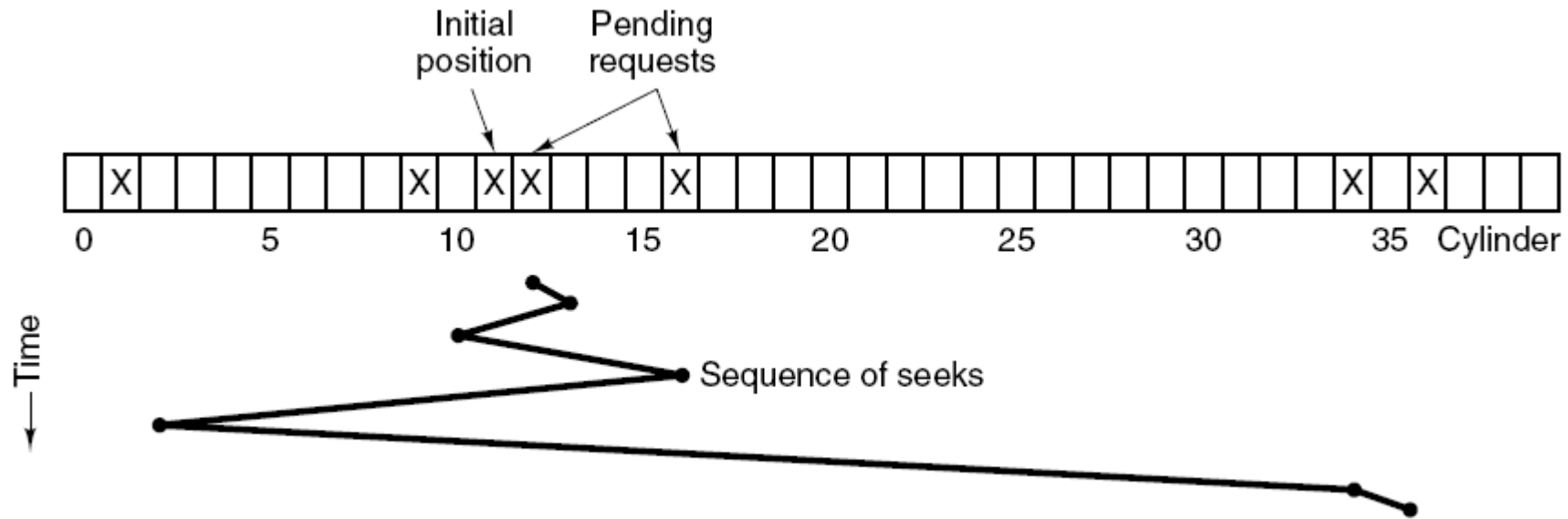
- Move the arm in one direction at a time
- Implication: must sort I/O requests by sector
- First write in ascending sector order, then write in descending sector order
- Still has fairness issues — what if you get a lot of requests close to the current point? (Remember that file systems try to allocate blocks locally)

Disk Arm Scheduling Algorithms (1)

Read/write time factors

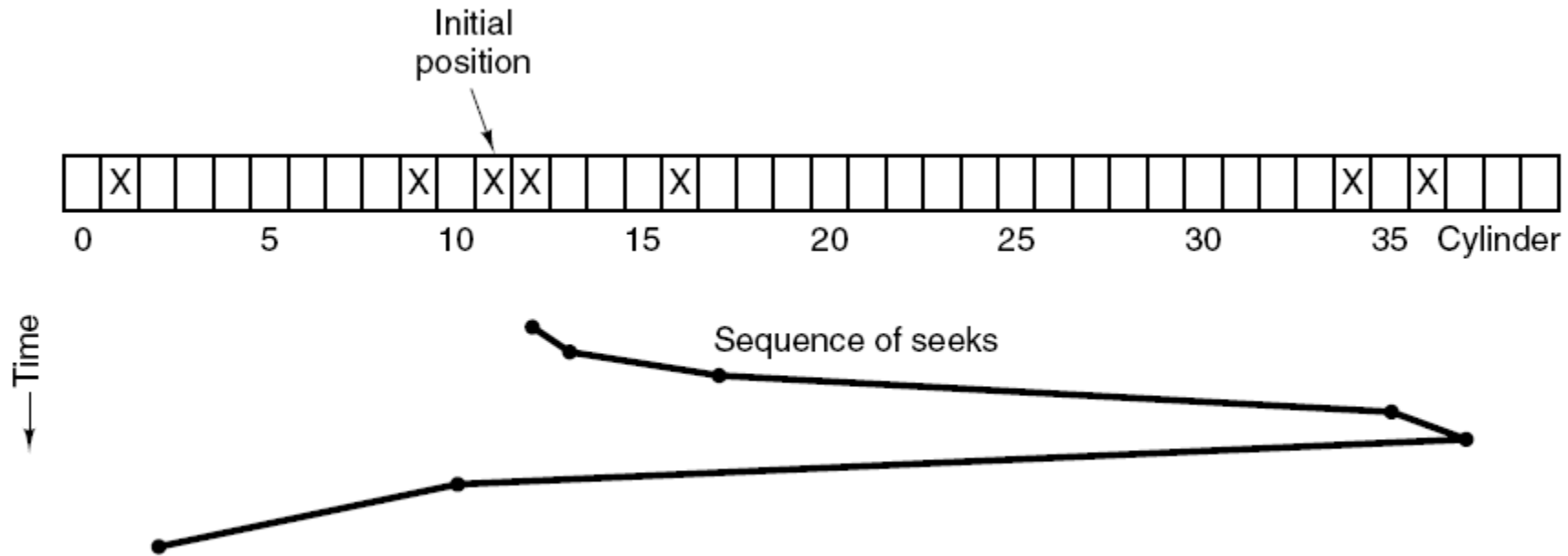
1. Seek time (the time to move the arm to the proper cylinder).
2. Rotational delay (the time for the proper sector to rotate under the head).
3. Actual data transfer time.

Disk Arm Scheduling Algorithms (2)



Shortest Seek First (SSF) disk scheduling algorithm

Disk Arm Scheduling Algorithms (3)



The elevator algorithm for scheduling disk requests

Order of Operations

- Recall that file system safety semantics demand that some blocks be written in a particular order
- This causes a discontinuity in the queue
- Besides, do you really know the disk geometry?
- The controller may implement its own version of this algorithm; the driver has to send it enough simultaneous requests that it can optimize

Rotational Delay

- Rotations take a while, too
- Scatter blocks around the disk to optimize for reading consecutive sectors
- Can be done at several layers: file system, device driver, controller
- Comments about disk geometry still apply

Graphics – A Simple Model

- The screen is just an array of pixels
- Map each pixel to some set of bits or bytes; have software calculate the value of each one
- Tell the screen the address of the array

Too Slow!

- That algorithm couldn't keep up with even modest graphics demands
- It wouldn't come close to being fast enough for games
- Must have hardware assist

Hardware Assist

- User programs work in higher-level terms: characters, polygons, filled or shaded areas, cursors, scrolling, etc.
- Each graphics controller is programmed — some use a standardized language, some a proprietary one
- Graphics processors are quite sophisticated (though specialized); they've been used for sorting and encryption
- OS must permit graphics driver to send high-level commands to device

Programming Interface

- Complex topic; out of scope for this class
- Always mediated through OS; screen is a shared resource
- Basic concept: the window
- Windows can be nested; application writes to a window are clipped at the window boundary
- The OS uses some screen area for itself (title bars, icons, etc.)

Network Devices

- Like disk drives, always shared among many processes
- However, no equivalent to directories or FATs
- Every input and output packet has to carry some identification (that's an oversimplification)

Network Output

- Output packets are relatively easy
- Mark packet depending on data from user's file descriptor
- Device drivers still have to adapt to local reality
- Ethernet: map 32-bit IP address to 48-bit Ethernet address
- Dial-up (PPP): must provide packet framing

Network Input

- Arbitrary incoming packets can be received
- Must use packet marking to associate packet with user process
- Different than disk: input is unsolicited
- Rate can be arbitrarily high, and you can't slow it down

The Network Stack

- Network processing involves many layers that provide different functions
- OS has to provide connections between the layers
- Unix System V had file system entries for each network device and for each protocol layer, plus commands to do the plumbing
- Most other systems use a different (non-filesystem) name space for network devices

Real I/O Issues

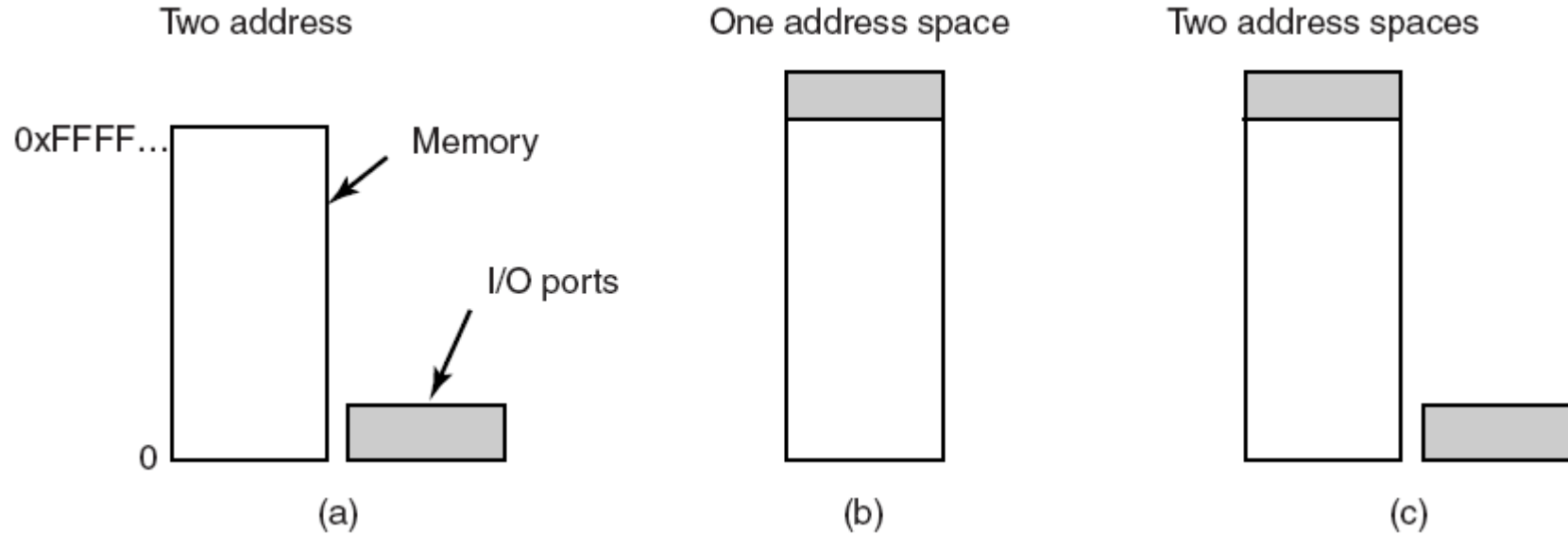
- Device drivers have to cope with physical world quirks
- More specifically, device drivers have to map from a simplified, abstract model of the world to whatever the processor and device present
- Many, many strange things out there. . .

The Linux Printer Driver

- Here are some comments from drivers/char/lp.c:

The so called 'buggy' handshake is really the well documented compatibility mode IEEE1284 handshake. They changed the well known Centronics handshake acking in the middle of busy expecting to not break drivers or legacy application, while they broken linux lp until I fixed it reverse engineering the protocol by hand some month ago...

Memory-Mapped I/O (1)

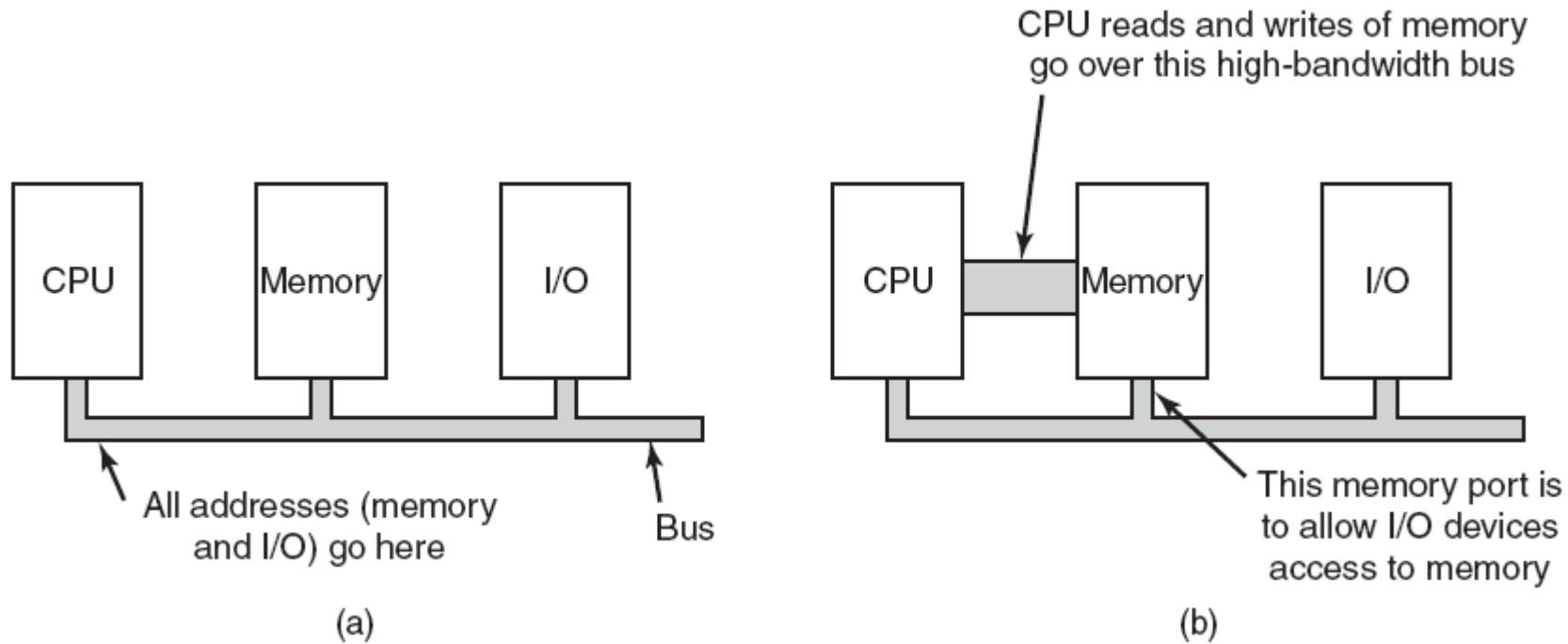


(a) Separate I/O and memory space.

(b) Memory-mapped I/O.

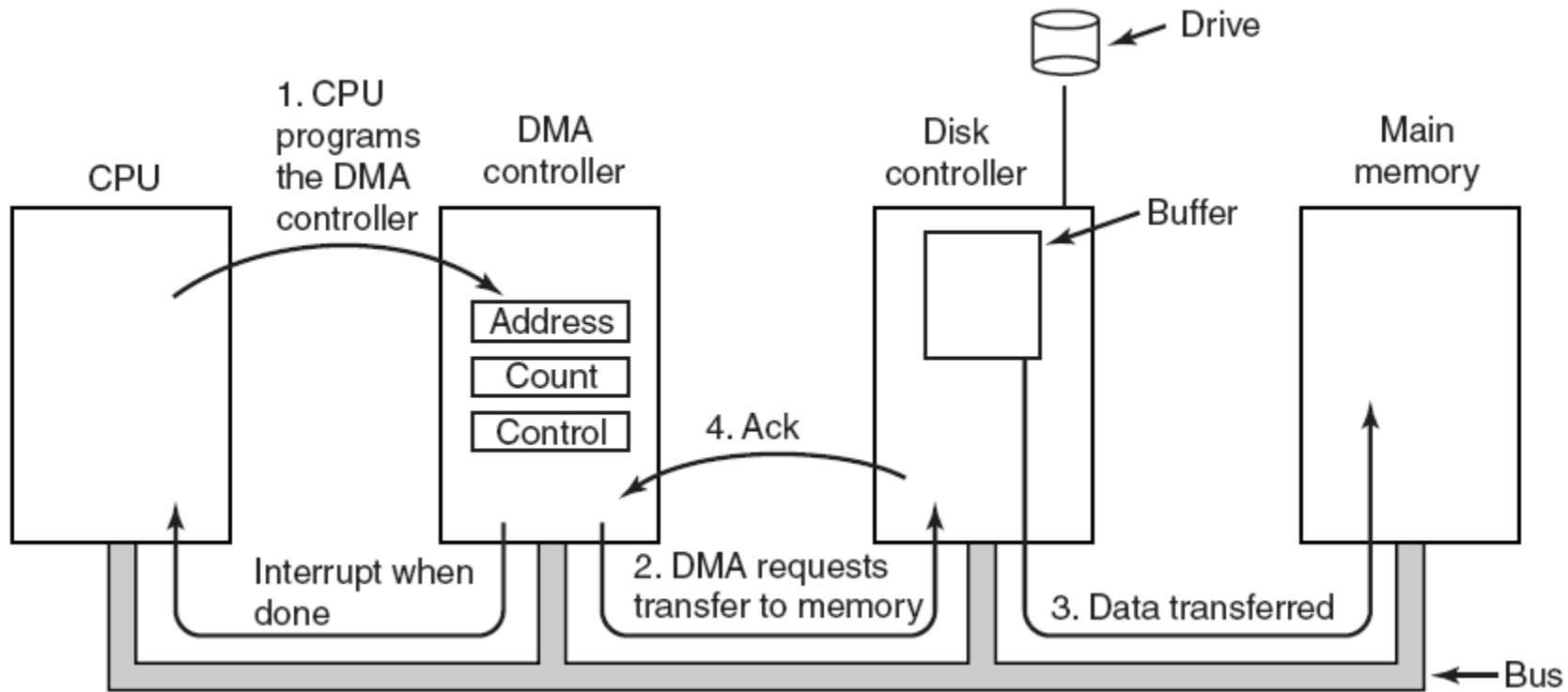
(c) Hybrid.

Memory-Mapped I/O (2)



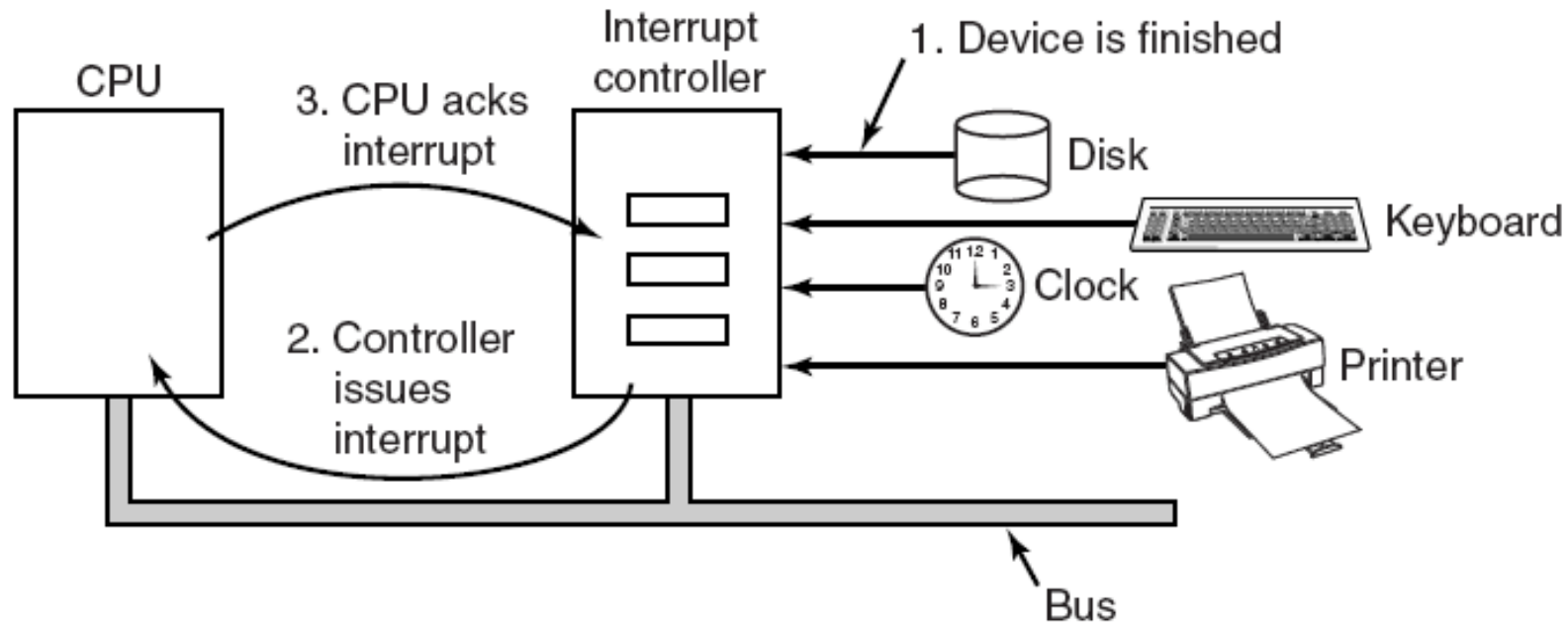
(a) A single-bus architecture.
(b) A dual-bus memory architecture.

Direct Memory Access (DMA)



Operation of a DMA transfer

Interrupts Revisited



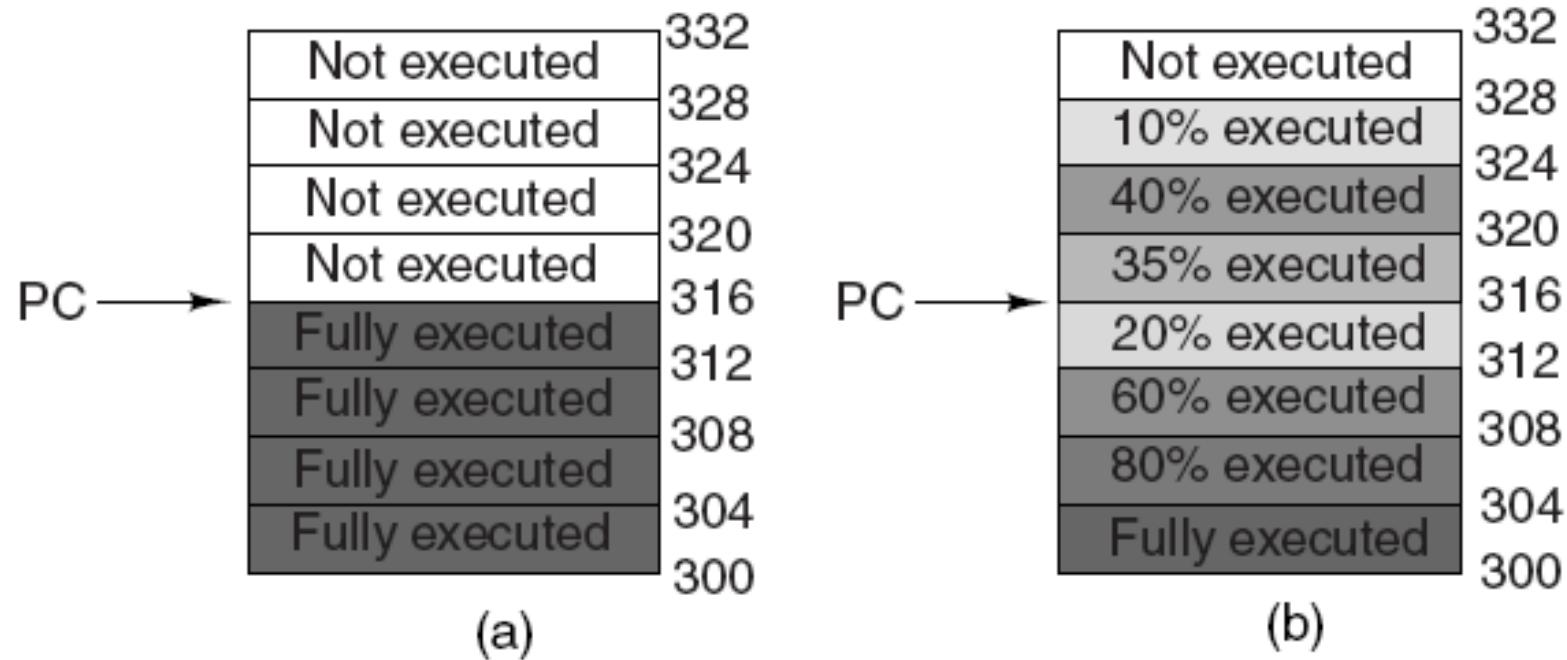
How an interrupt happens. The connections between the devices and the interrupt controller actually use interrupt lines on the bus rather than dedicated wires.

Precise and Imprecise Interrupts (1)

Properties of a *precise interrupt*

1. PC (Program Counter) is saved in a known place.
2. All instructions before the one pointed to by the PC have fully executed.
3. No instruction beyond the one pointed to by the PC has been executed.
4. Execution state of the instruction pointed to by the PC is known.

Precise and Imprecise Interrupts (2)



(a) A precise interrupt.
(b) An imprecise interrupt.

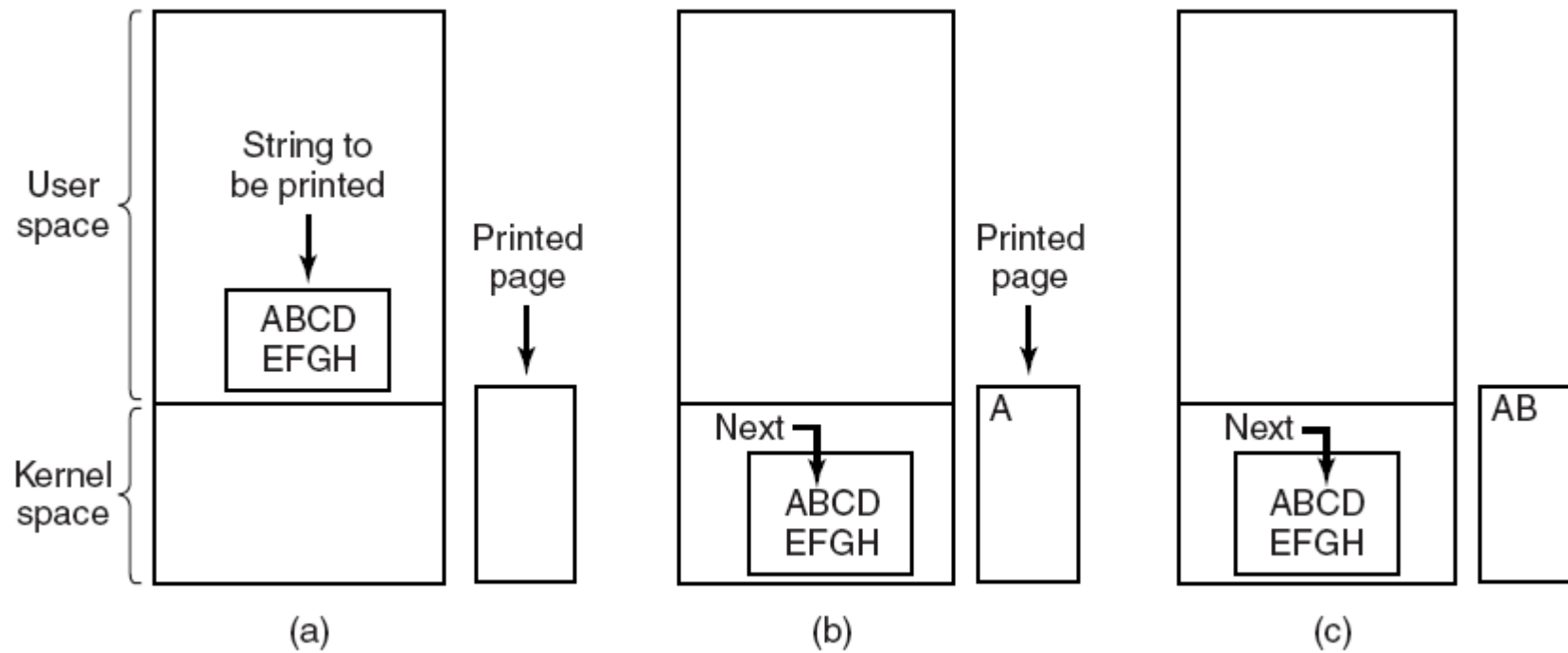
Interrupt Loads

- Normally, you start an I/O operation and return; an interrupt occurs when it's complete
- But processing I/O interrupts is expensive
- For high input rates, adapt dynamically; configure device to interrupt every n^{th} input event
- For output, it may be cheaper to poll in a spin loop, then sleep — via a timer — if the output FIFO is full

Programmed I/O and DMA

- Simple, low-speed devices operate by programmed I/O: the CPU copies a byte at a time to the device
- Higher-speed devices use Direct Memory Access (DMA): the CPU passes the device an address and a count, and it copies the data directly from memory
- Sometimes called cycle-stealing: the device steals bus cycles from the CPU

Programmed I/O (1)



Steps in printing a string.

Programmed I/O (2)

```
copy_from_user(buffer, p, count);           /* p is the kernel buffer */
for (i = 0; i < count; i++) {                /* loop on every character */
    while (*printer_status_reg != READY);    /* loop until ready */
    *printer_data_register = p[i];           /* output one character */
}
return_to_user();
```

Writing a string to the printer using programmed I/O

Interrupt-Driven I/O

```
copy_from_user(buffer, p, count);
enable_interrupts();
while (*printer_status_reg != READY) ;
*printer_data_register = p[0];
scheduler();
```

(a)

```
if (count == 0) {
    unblock_user();
} else {
    *printer_data_register = p[i];
    count = count - 1;
    i = i + 1;
}
acknowledge_interrupt();
return_from_interrupt();
```

(b)

- Writing a string to the printer using interrupt-driven I/O.
- (a) Code executed at the time the print system call is made.
 - (b) Interrupt service procedure for the printer.

I/O Using DMA

```
copy_from_user(buffer, p, count);  
set_up_DMA_controller( );  
scheduler();
```

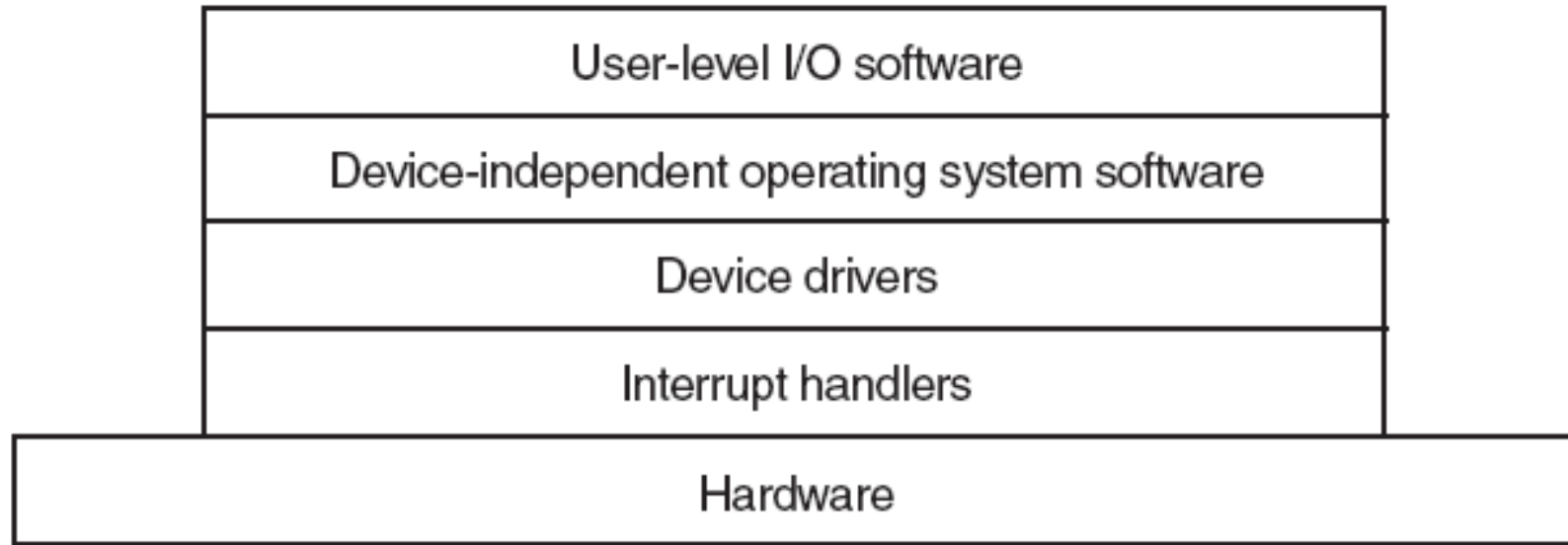
(a)

```
acknowledge_interrupt( );  
unblock_user( );  
return_from_interrupt( );
```

(b)

Printing a string using DMA. (a) Code executed when the print system call is made.
(b) Interrupt service procedure.

I/O Software Layers



Layers of the I/O software system

Interrupt Handlers (1)

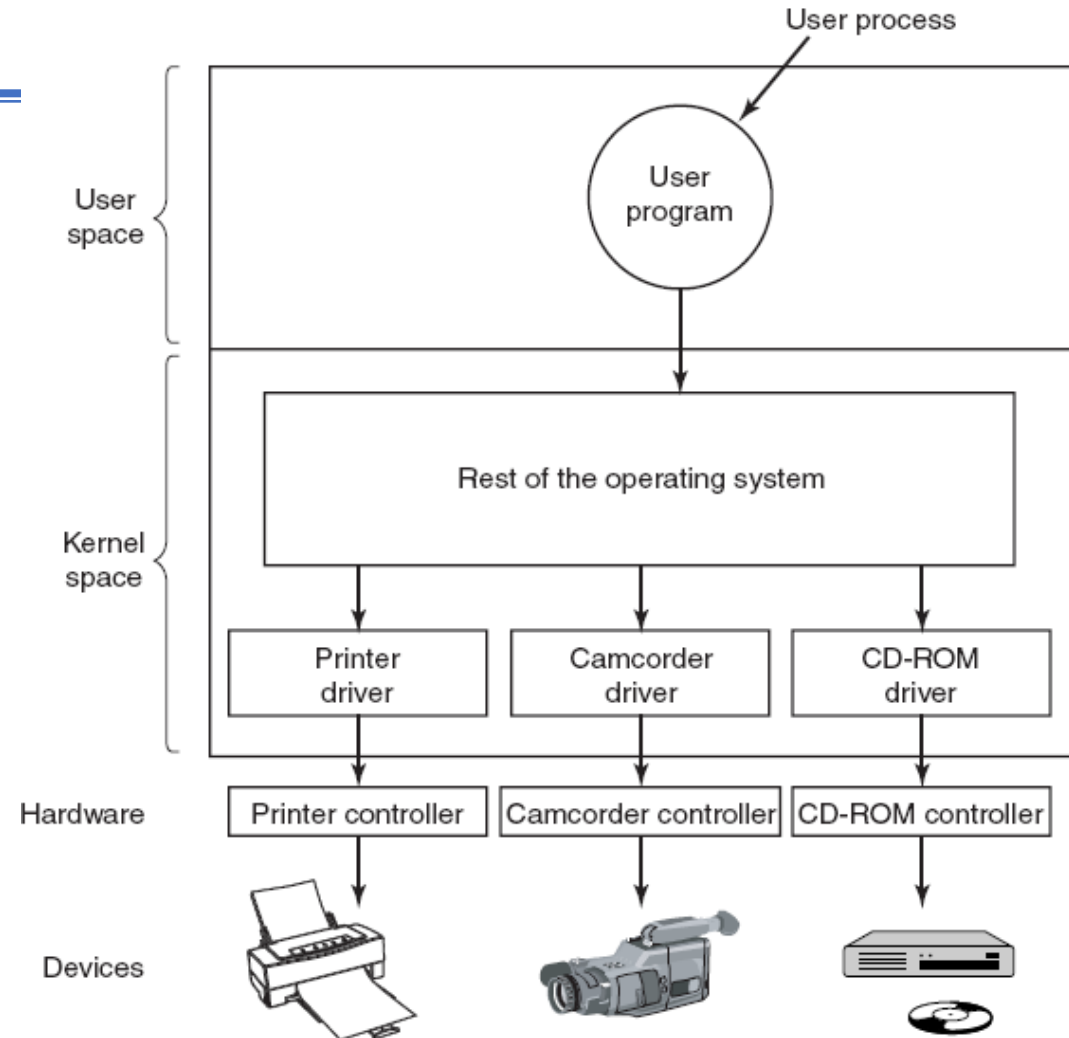
1. Save registers not already been saved by interrupt hardware.
2. Set up a context for the interrupt service procedure.
3. Set up a stack for the interrupt service procedure.
4. Acknowledge the interrupt controller. If there is no centralized interrupt controller, reenables interrupts.
5. Copy the registers from where they were saved to the process table.

Interrupt Handlers (2)

- 6.Run the interrupt service procedure.
- 7.Choose which process to run next.
- 8.Set up the MMU context for the process to run next.
- 9.Load the new process' registers, including its PSW.
- 10.Start running the new process.

Device Drivers

Logical positioning
of device drivers. In
reality all
communication
between drivers
and device
controllers goes
over the bus

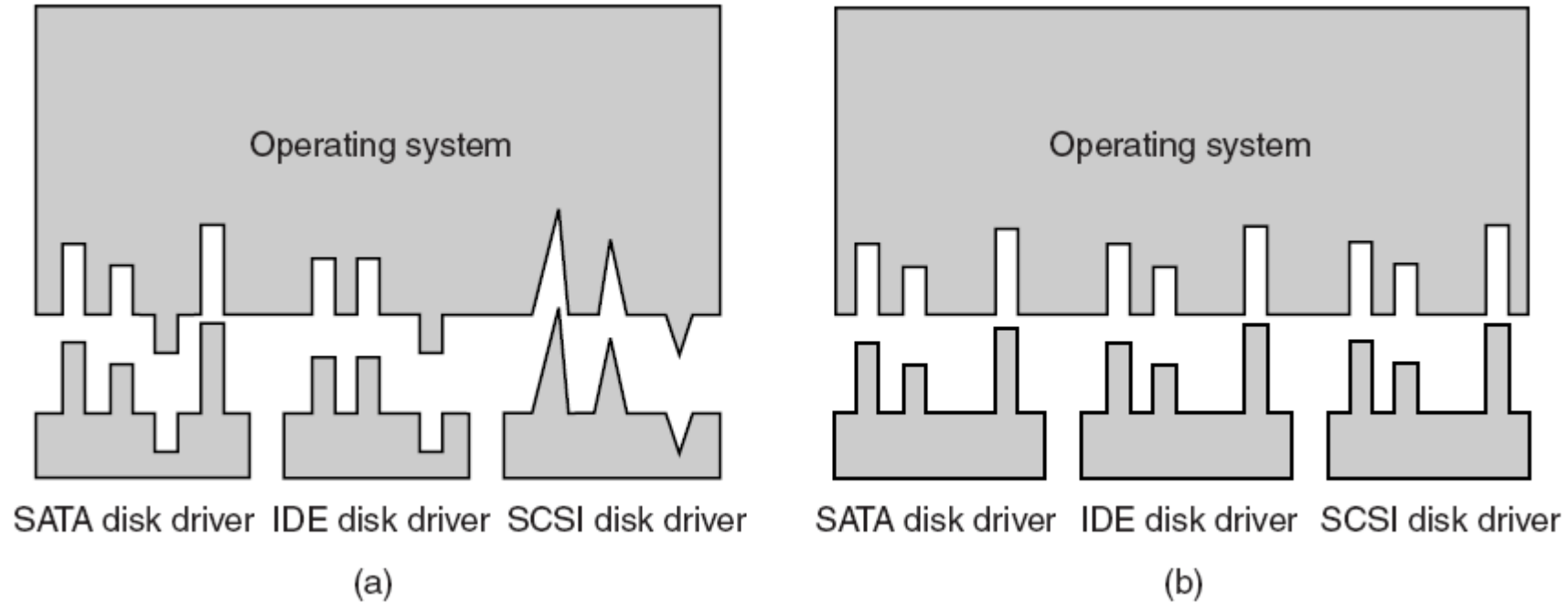


Device-Independent I/O Software

Uniform interfacing for device drivers
Buffering
Error reporting
Allocating and releasing dedicated devices
Providing a device-independent block size

Functions of the device-independent I/O software

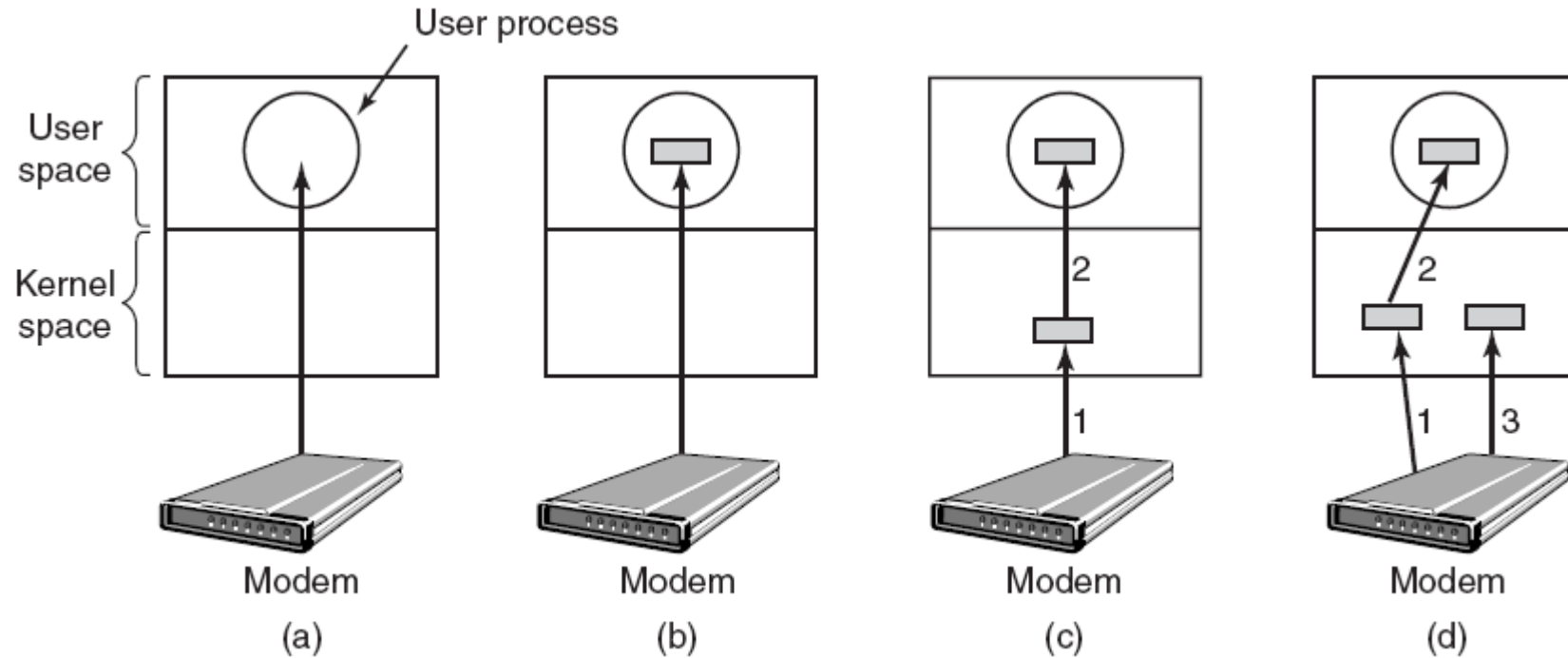
Uniform Interfacing for Device Drivers



(a) Without a standard driver interface.

(b) With a standard driver interface.

Buffering (1)



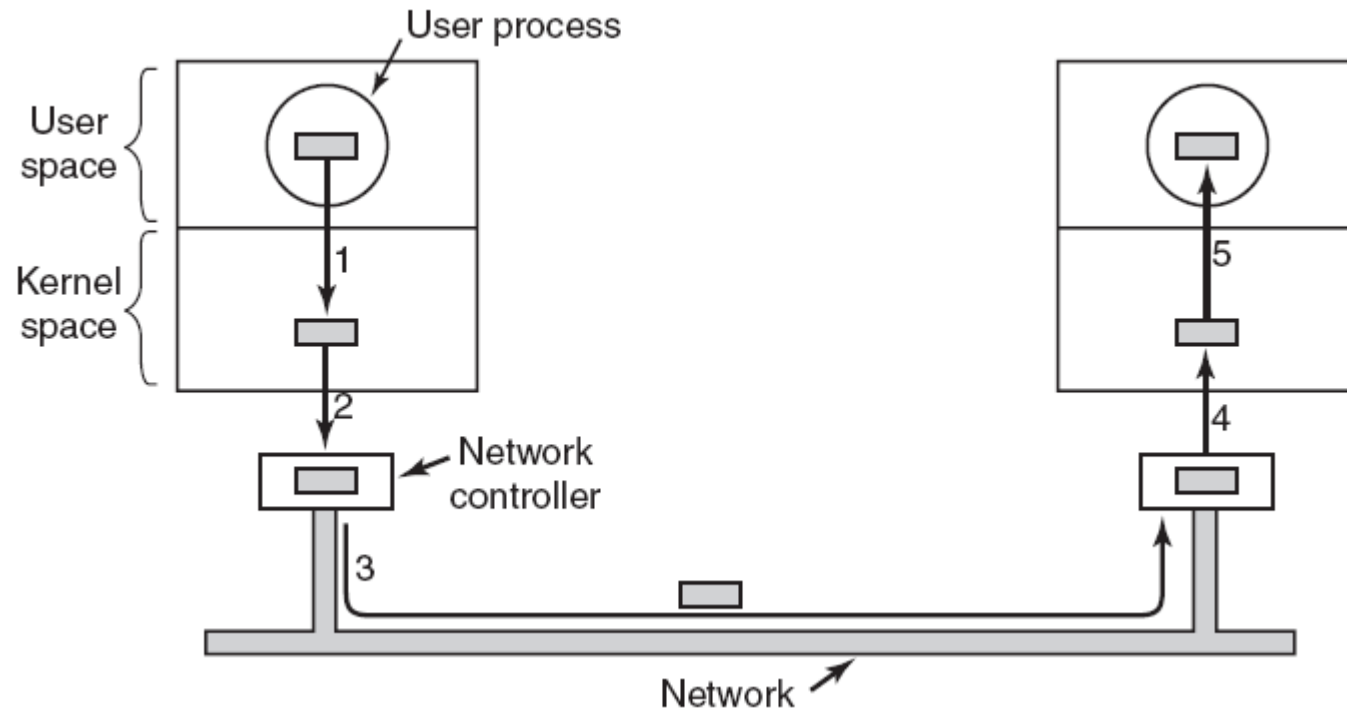
(a) Unbuffered input.

(b) Buffering in user space.

(c) Buffering in the kernel followed by copying to user space.

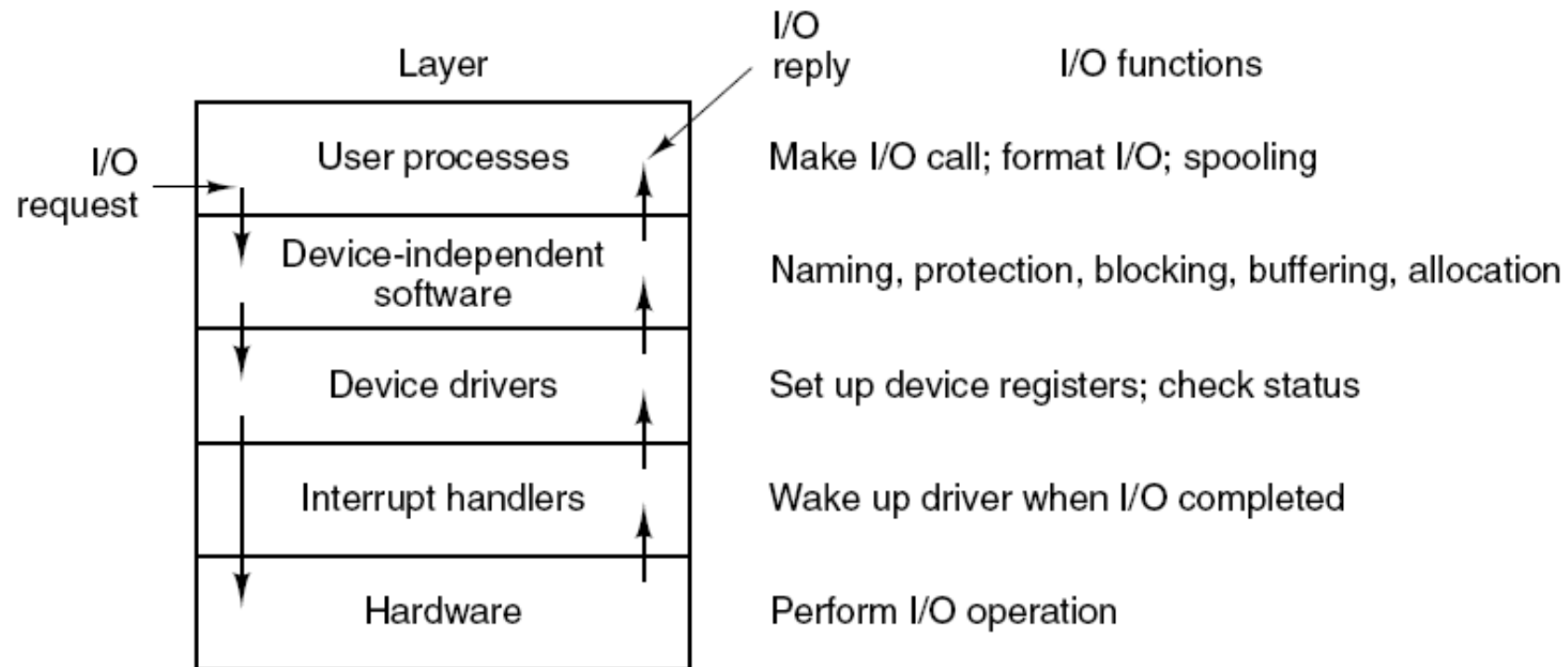
(d) Double buffering in the kernel.

Buffering (2)



Networking may involve many copies of a packet

User-Space I/O Software



Layers of the I/O system and the main functions of each layer

Memory Access Issues

- Cache consistency
- Virtual memory
- Direct I/O from user space
- Address space size

Virtual Memory

- The kernel is dealing with (kernel) virtual addresses
- These do not correspond to physical addresses
- Contiguous virtual addresses are probably not contiguous in physical memory
- Some systems have an I/O map — the I/O bus manager has a (version of) the virtual memory map
- More often, the kernel has to translate the virtual addresses itself

Scatter/Gather I/O

- Suppose we're reading a single packet or disk block into two or more non-contiguous pages
- The I/O transfer has to use more than one $\{haddress, length_i\}$ pair for that transfer to scatter the data around memory
- The same applies on output, where it has to be gathered from different physical pages

Direct I/O

- For efficiency, we may want to avoid copying data to/from user space
- Sometimes possible to do direct I/O
- Must consult user virtual memory map for translation
- Must lock pages in physical memory during I/O

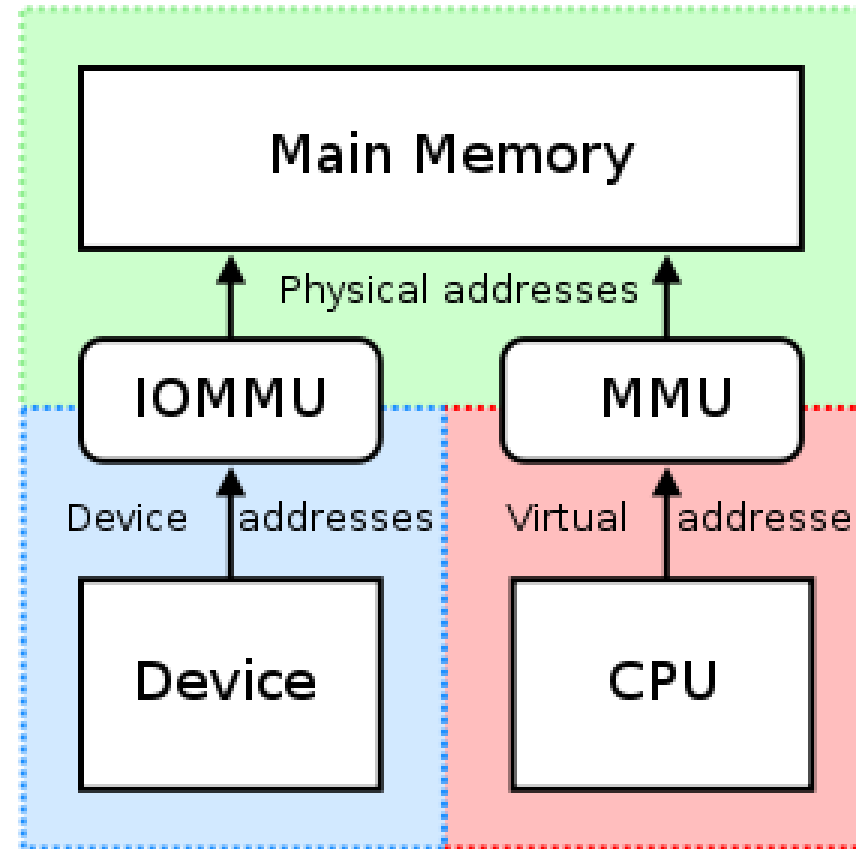
Address Space Size

- Older I/O devices and buses may not be able to address all of system memory
- Example: ISA bus has a 16-bit address register: 64K bytes
- Solution: bounce buffers
- Do I/O to permissible memory — typically, permissible physical memory — then copy the page
- Usually, the page is copied, not remapped – we need that precious low memory space

Disk Block Address Space

- Others made similar mistakes: they didn't anticipate the growth of disk sizes
- Some historical disk size limits:
 - 504M maximum value for CHS
 - 1.97G 12-bit cylinder size limit in BIOS
 - 2G Largest FAT16 disk
 - 3.94G With 16 heads, can't have more than 8K cylinders
 - 7.38G To work around the 8K cylinder limit, can't have more than 240 heads
 - 7.88G BIOS can't address more than 7.88G with CHS addressing; must use LBA
 - 128G disk spec uses 28-bit number for sector
 - 2.2T Windows XP limit

IOMMU (for DMA Capable IO Devices)



The IOMMU maps device-visible virtual addresses (also called *device addresses* or *I/O addresses* in this context) to (possibly fragmented) physical addresses

Wikipedia

IOMMU Advantages

- Large regions of memory can be allocated without the need to be contiguous in physical memory – the IOMMU maps contiguous virtual addresses to the underlying fragmented physical addresses. Thus, the use of vectored I/O (scatter-gather lists) can sometimes be avoided.
- Devices that do not support memory addresses long enough to address the entire physical memory can still address the entire memory through the IOMMU, avoiding overheads associated with copying buffers to and from the peripheral's addressable memory space.
 - For example, x86 computers can address more than 4 gigabytes of memory with the Physical Address Extension (PAE) feature in an x86 processor. Still, an ordinary 32-bit PCI device simply cannot address the memory above the 4 GiB boundary, and thus it cannot directly access it. Without an IOMMU, the operating system would have to implement time-consuming bounce buffers (also known as double buffers[3]).
- Memory is protected from malicious devices that are attempting DMA attacks and faulty devices that are attempting errant memory transfers because a device cannot read or write to memory that has not been explicitly allocated (mapped) for it. The memory protection is based on the fact that OS running on the CPU (see figure) exclusively controls both the MMU and the IOMMU. The devices are physically unable to circumvent or corrupt configured memory management tables.

IOMMU Disadvantages

- Some degradation of performance from translation and management overhead (e.g., page table walks)
- Consumption of physical memory for the added I/O page (translation) tables. This can be mitigated if the tables can be shared with the processor

Error Handling

- I/O devices and physical media aren't perfect
- Device drivers have to cope with error conditions
- Recovery can be complex and device-dependent
- Often, errors are detected by adding check bytes or ECC bytes to physical blocks

Example: Tape Errors

- Suppose a tape drive indicated a bad read
- First, retry up to 10 times
- Backspace the tape a few inches and “shoe shine” it — move it back and forth so the bad spot is scraped against the edge of the read head
- The idea is to dislodge any pieces of dirt

Disk I/O Errors

- If a disk write fails, retry a few times
- If the write still fails, mark the block as “bad” — how this is done is device-dependent — and allocate a spare block
- Somehow, indicate that this remapping has taken place

Testing for Errors

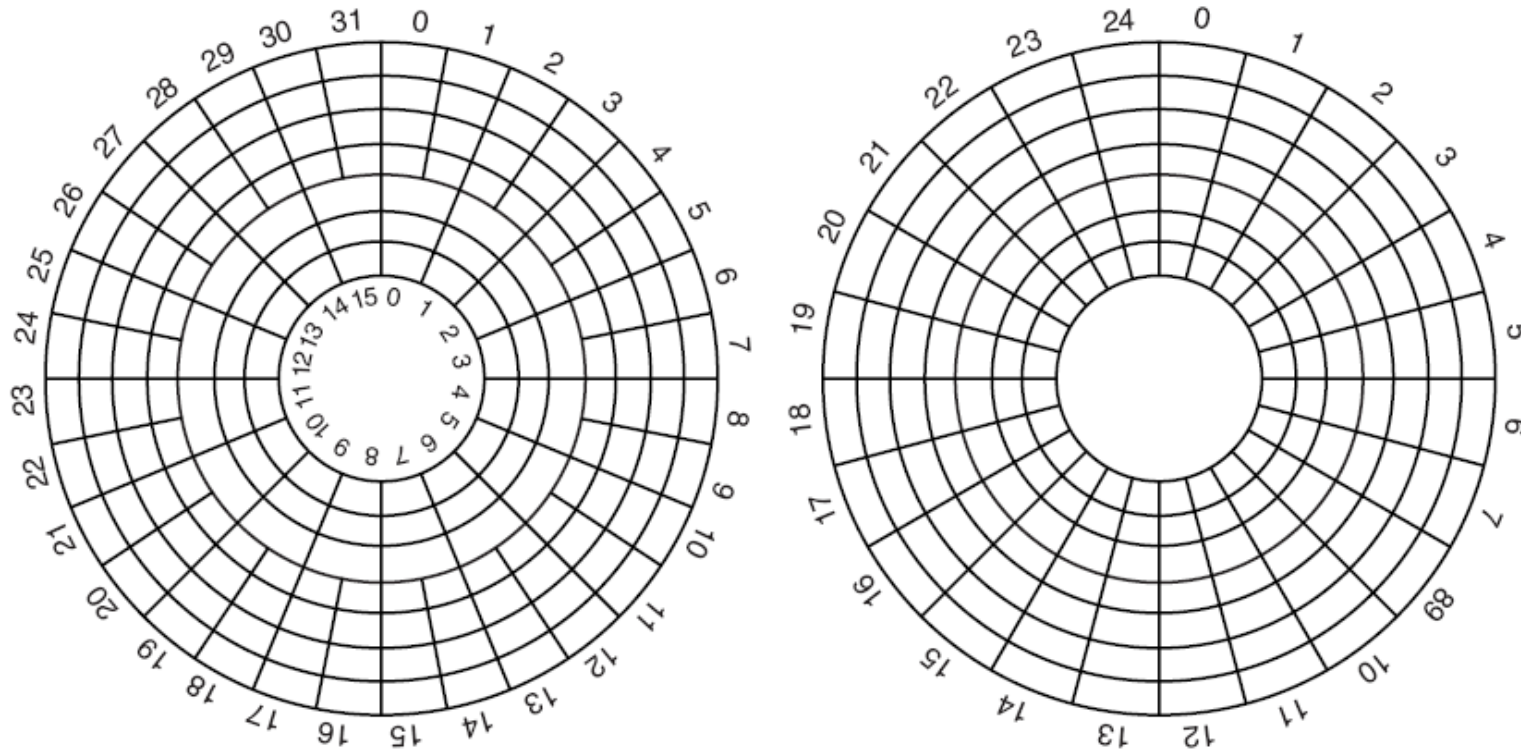
- How do you test your error recovery code?
- Where do you get media that are just bad enough?
- Remember that failure modes can be arbitrarily weird
- Loose cables can generate very strange signals

Magnetic Disks (1)

Parameter	IBM 360-KB floppy disk	WD 18300 hard disk
Number of cylinders	40	10601
Tracks per cylinder	2	12
Sectors per track	9	281 (avg)
Sectors per disk	720	35742000
Bytes per sector	512	512
Disk capacity	360 KB	18.3 GB
Seek time (adjacent cylinders)	6 msec	0.8 msec
Seek time (average case)	77 msec	6.9 msec
Rotation time	200 msec	8.33 msec
Motor stop/start time	250 msec	20 sec
Time to transfer 1 sector	22 msec	17 μ sec

Disk parameters for the original IBM PC 360-KB floppy disk and a Western Digital WD 18300 hard disk

Magnetic Disks (2)



(a) Physical geometry of a disk with two zones.

(b) A possible virtual geometry for this disk.

From the NetBSD Printer Driver

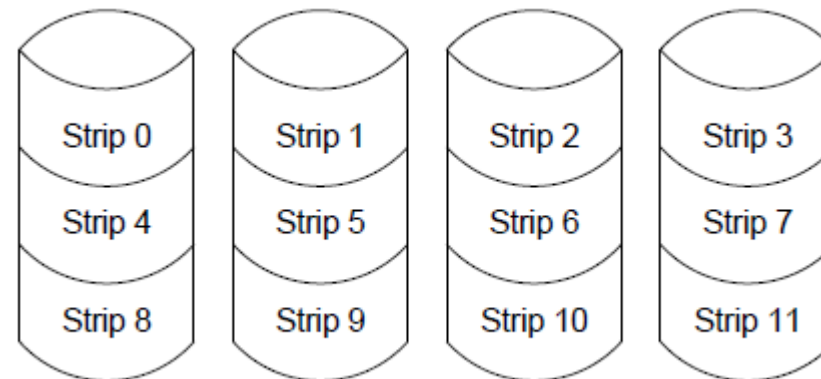
1. You should be able to write to and read back the same value to the data port. Do an alternating zeros, alternating ones, walking zero, and walking one test to check for stuck bits.
2. You should be able to write to and read back the same value to the control port lower 5 bits, the upper 3 bits are reserved per the IBM PC technical reference manuals and different boards do different things with them. Do an alternating zeros, alternating ones, walking zero, and walking one test to check for stuck bits.
3. ...

RAID

- One way of coping with errors: redundancy
- Example: RAID (Redundant Array of Inexpensive (or Independent) Disks)
- Many configurations
- Some use extra disks for error recovery, some use them for better performance, some do both
- Can be implemented in controller or OS
- In OS, implemented as pseudo-device driver layerd on top of real devices

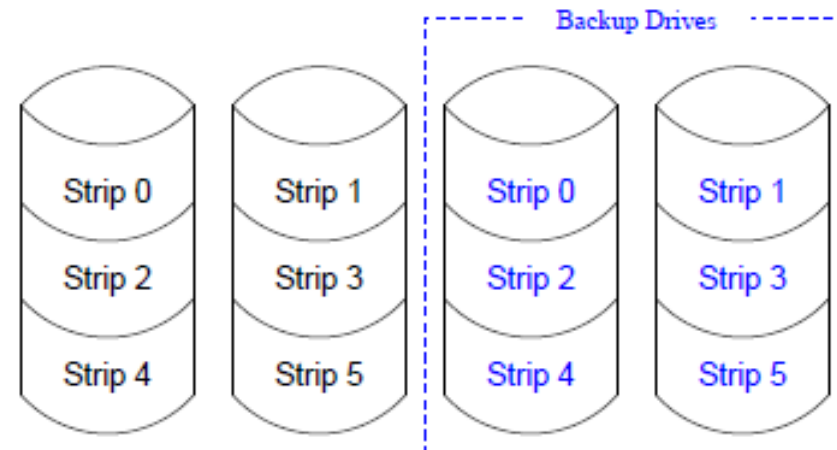
RAID 0

- Put strips of sectors on successive disks — called striping
- Improves throughput — can be doing I/O from multiple drives simultaneously to satisfy a single (large) request
- Parallelism improves with more drives
- But — hurts reliability



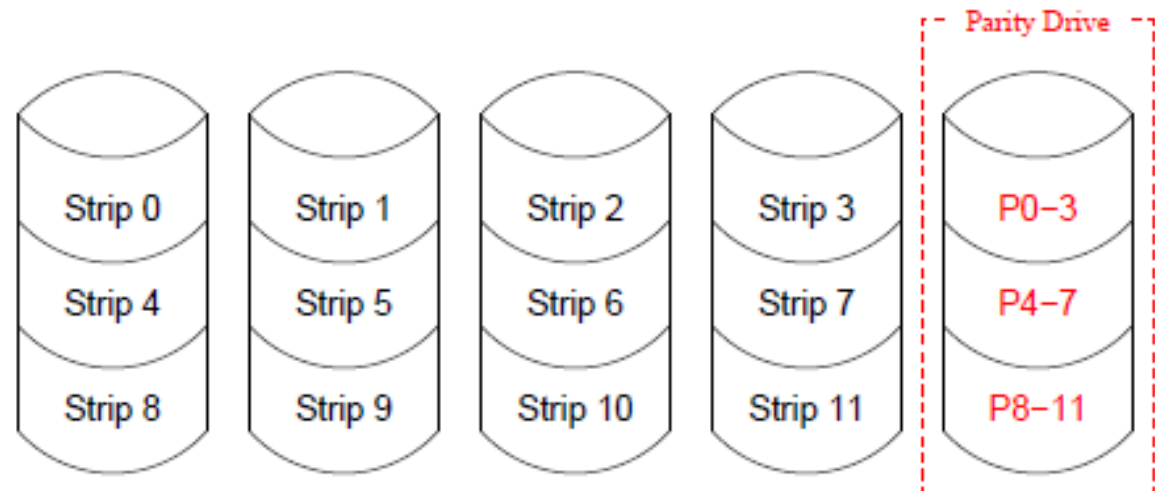
RAID 1

- Replicate each sector; write both copies
- Improves read throughput; hurts write throughput
- If a drive fails, ignore it and use the other one
- Rebuild the failed drive in the background



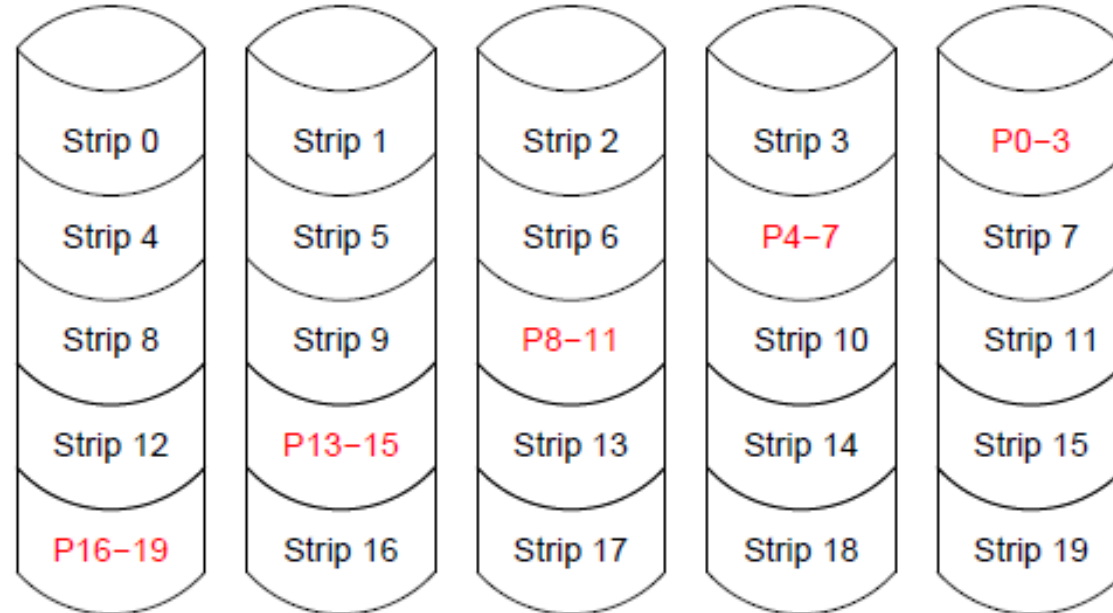
RAID 4

- The last drive is the parity drive — it has the XOR of all of the other drives
- If a drive crashes, reconstruct its contents from all of the other drives, including the parity drive
- Poor performance — must read old data to recompute parity if a block changes
- Heavy load on parity drive

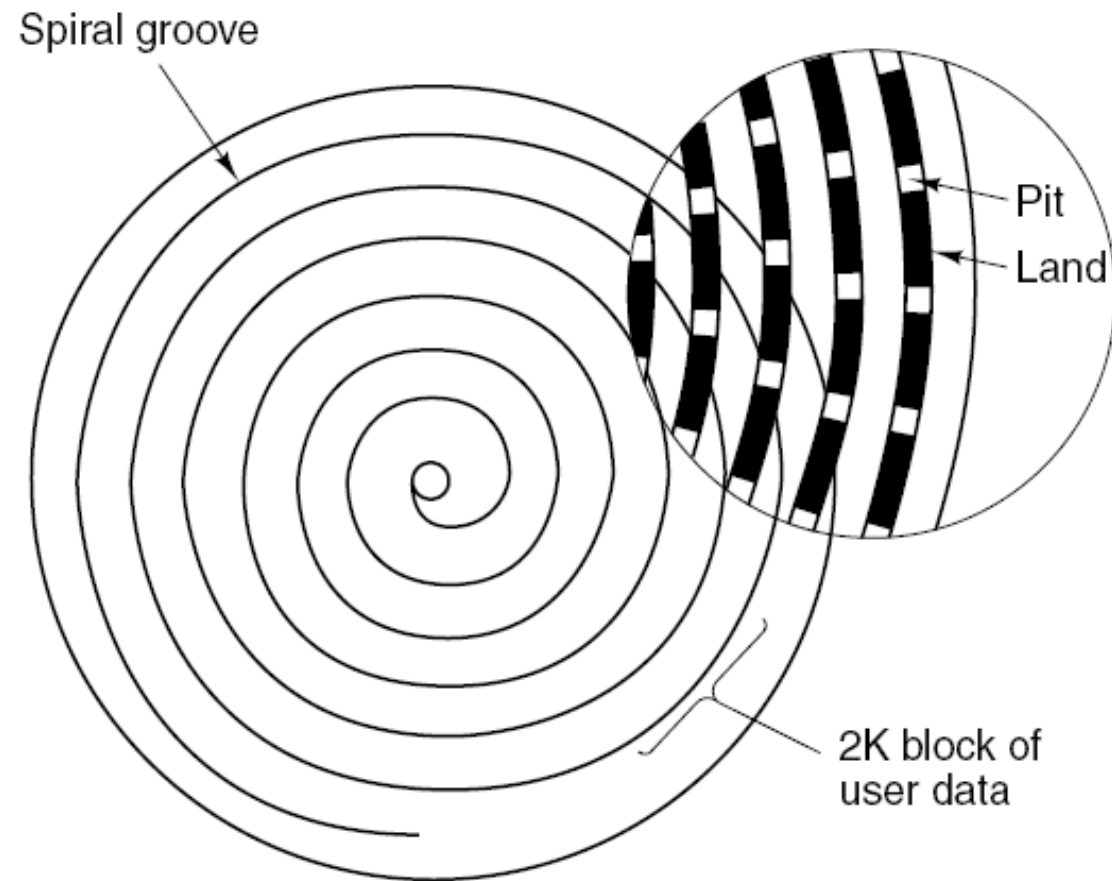


RAID 5

- Similar to RAID 4, but the parity strips are spread out
- Avoids bottleneck
- But — rebuilding after a drive failure is complex

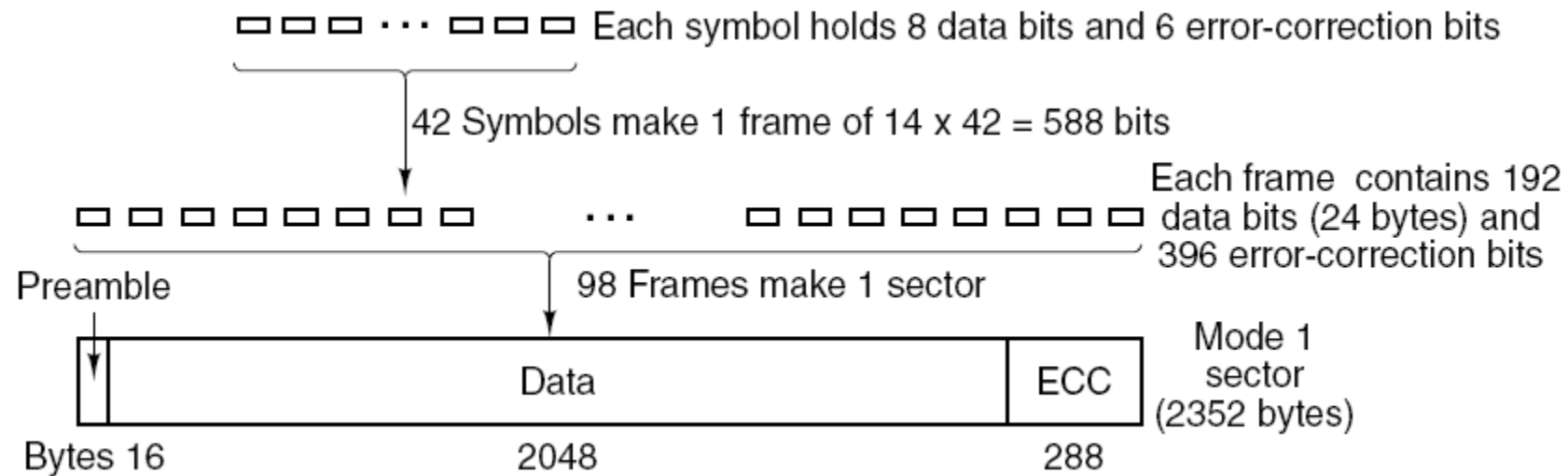


CD-ROMs (1)



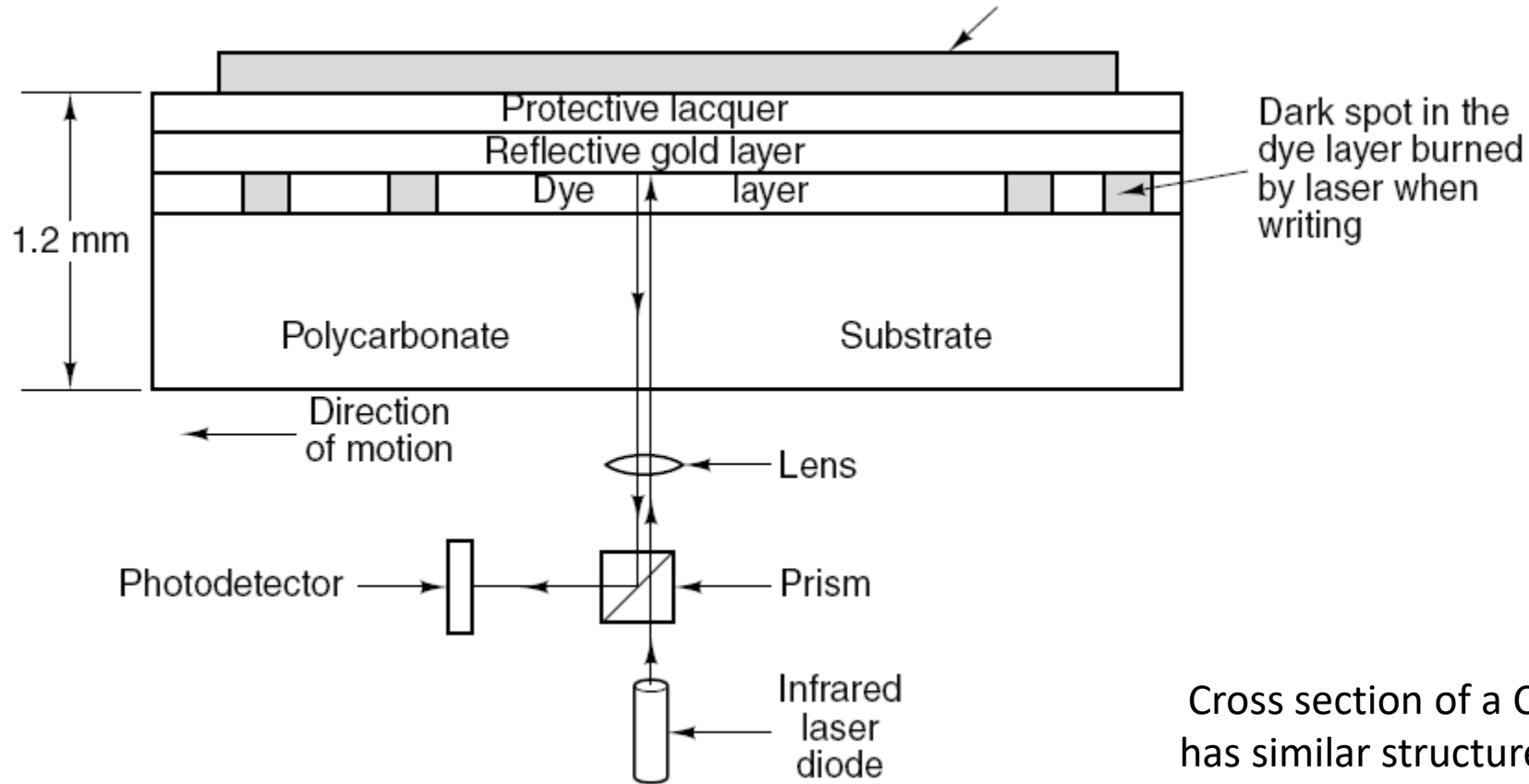
Recording structure of a compact disc or CD-ROM.

CD-ROMs (2)



Logical data layout on a CD-ROM.

CD-Recordables (1)



Cross section of a CD-R disk and laser. A silver CD-ROM has similar structure, except without dye layer and with pitted aluminum layer instead of gold layer.

DVD (1)

DVD Improvements on CDs

1. Smaller pits

(0.4 microns versus 0.8 microns for CDs).

2. A tighter spiral

(0.74 microns between tracks versus 1.6 microns for CDs).

3. A red laser

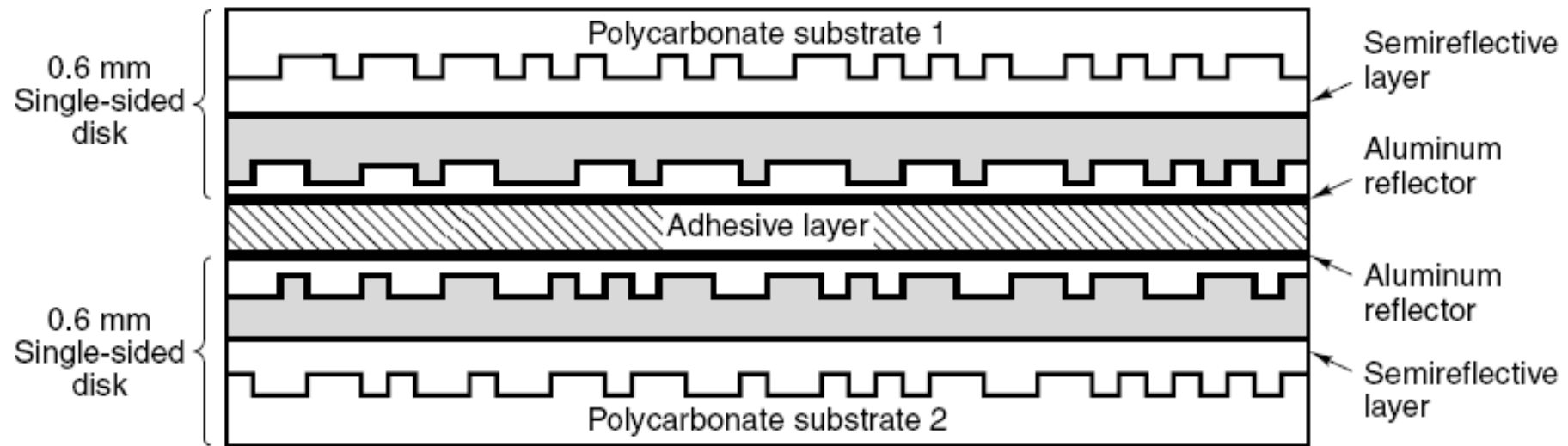
(at 0.65 microns versus 0.78 microns for CDs).

DVD (2)

DVD Formats

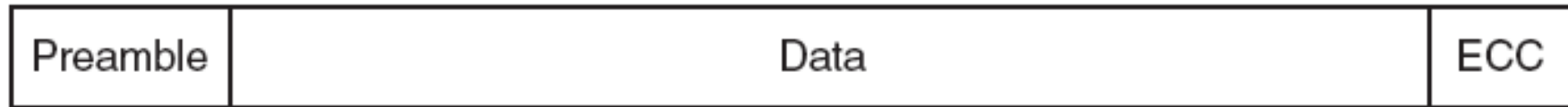
1. Single-sided, single-layer (4.7 GB).
2. Single-sided, dual-layer (8.5 GB).
3. Double-sided, single-layer (9.4 GB).
4. Double-sided, dual-layer (17 GB).

DVD (3)



A double-sided, dual-layer DVD disk.

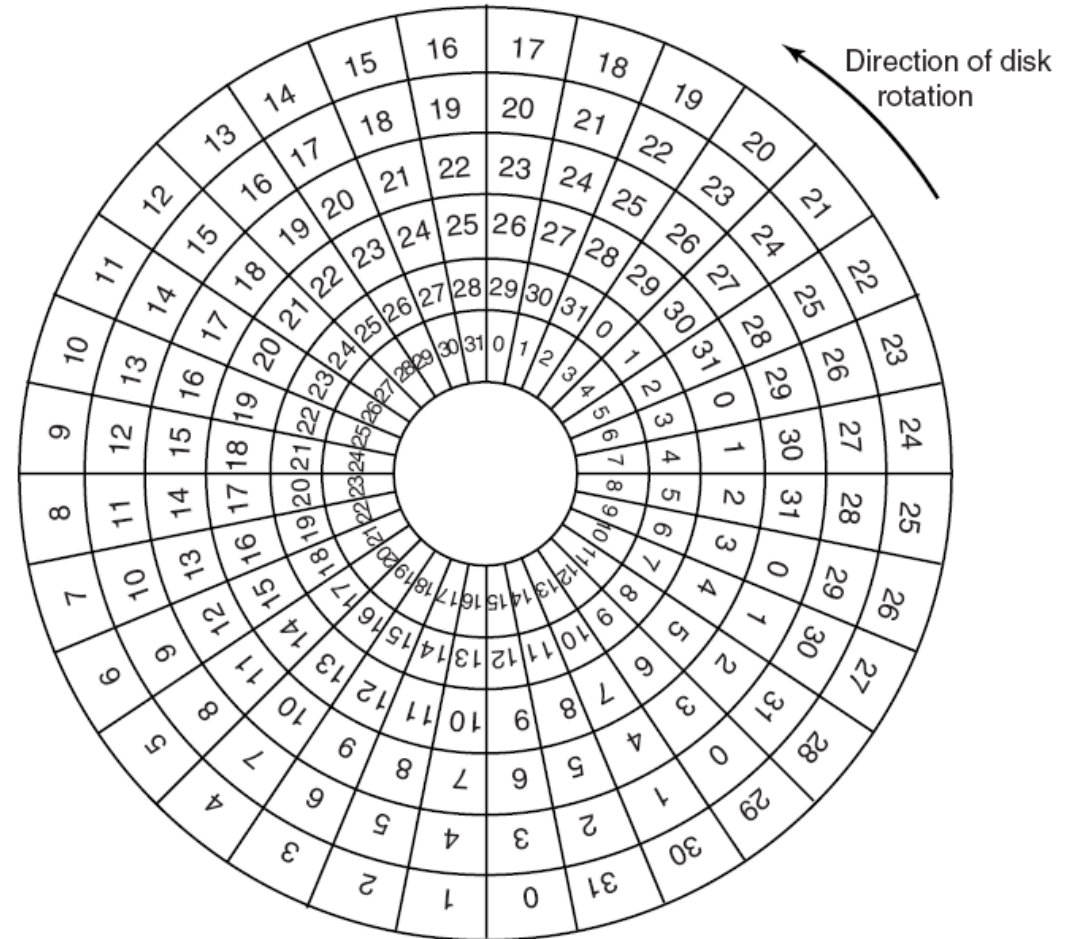
Disk Formatting (1)



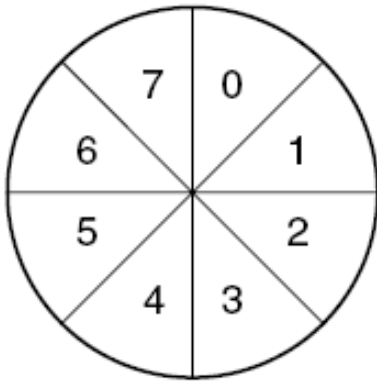
A disk sector.

Disk Formatting (2)

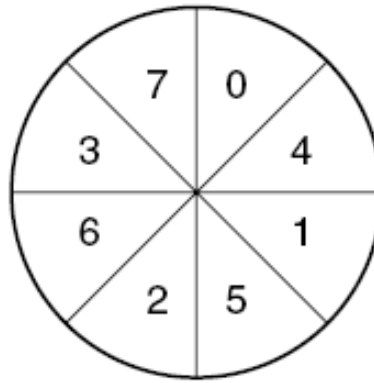
An illustration of cylinder skew



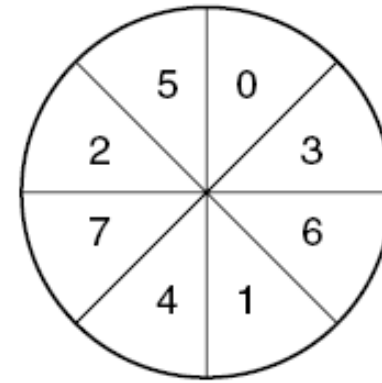
Disk Formatting (3)



(a)



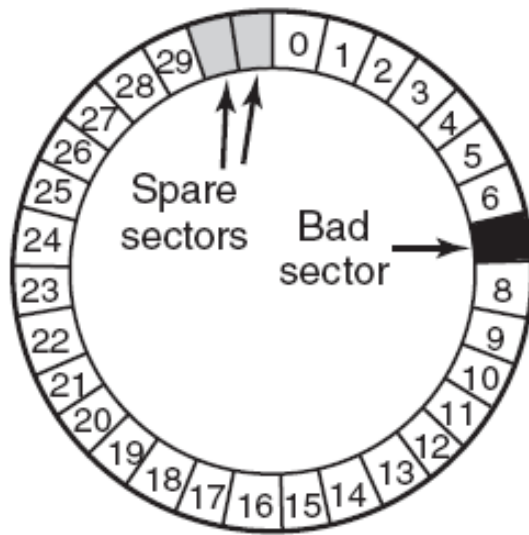
(b)



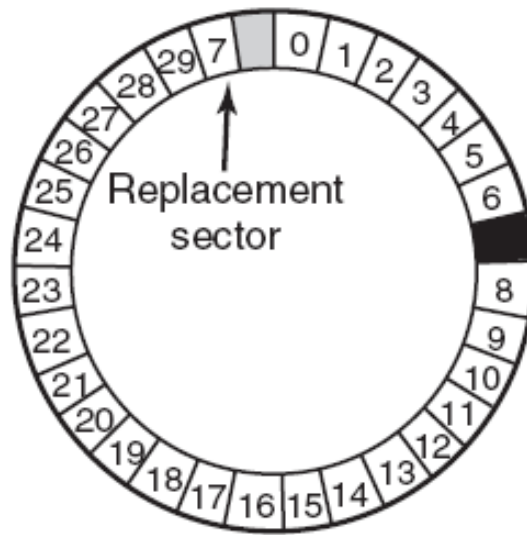
(c)

- (a) No interleaving.
- (b) Single interleaving.
- (c) Double interleaving.

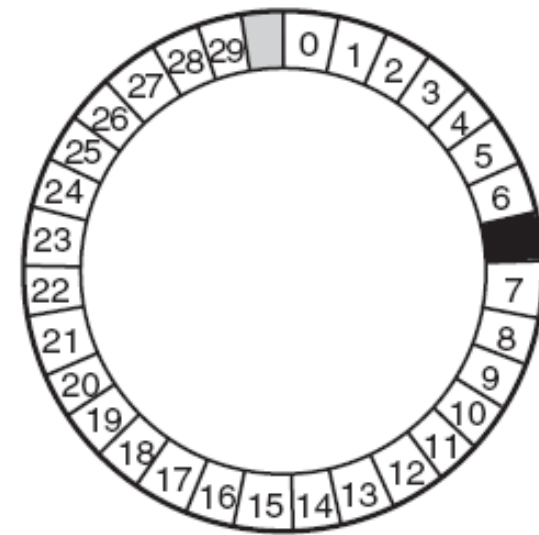
Error Handling



(a)



(b)



(c)

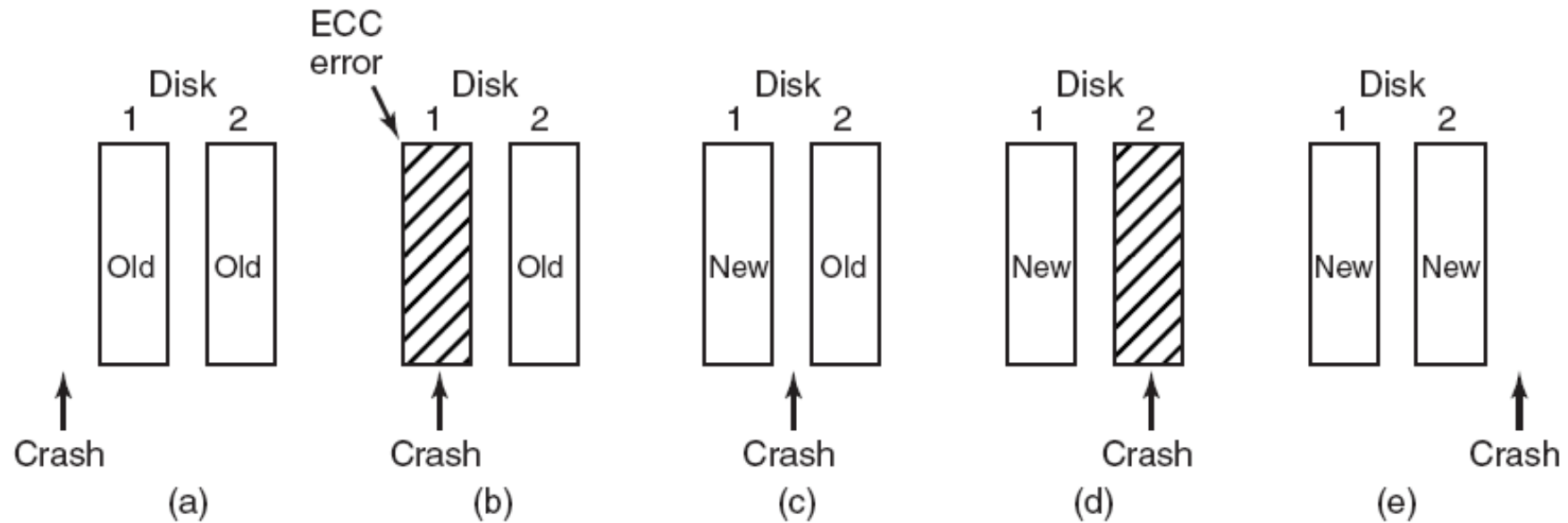
- (a) A disk track with a bad sector.
- (b) Substituting a spare for the bad sector.
- (c) Shifting all the sectors to bypass the bad one.

Stable Storage (1)

Operations for stable storage using identical disks:

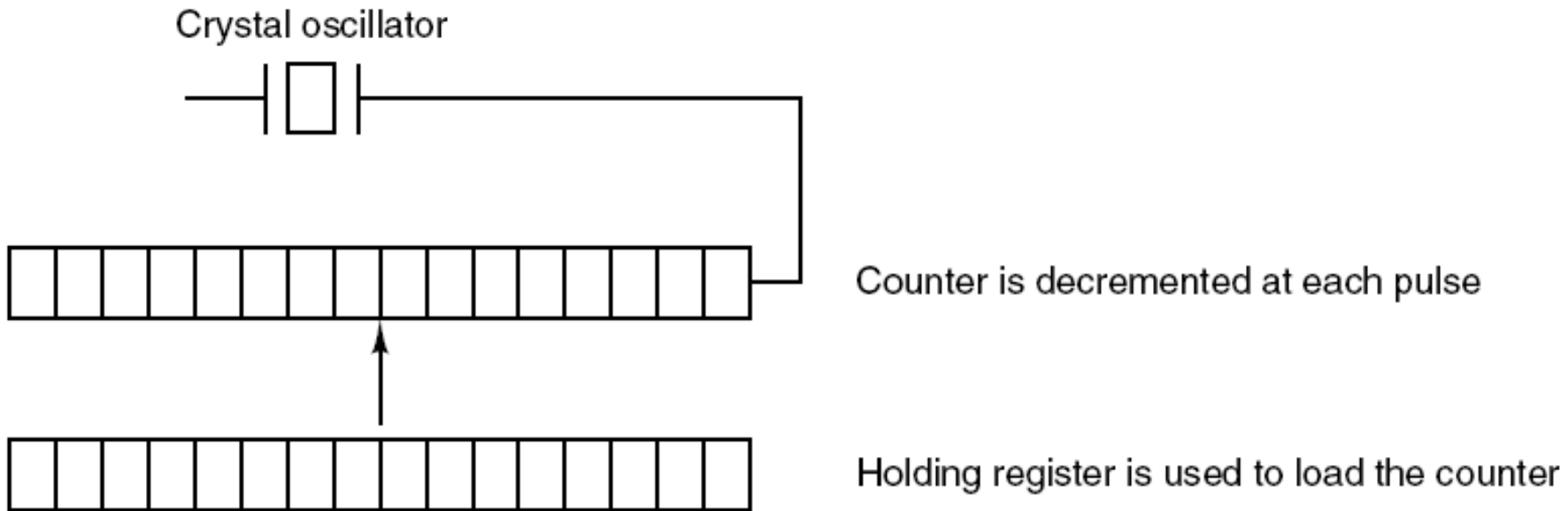
1. Stable writes
2. Stable reads
3. Crash recovery

Stable Storage (2)



Analysis of the influence of crashes on stable writes

Clock Hardware



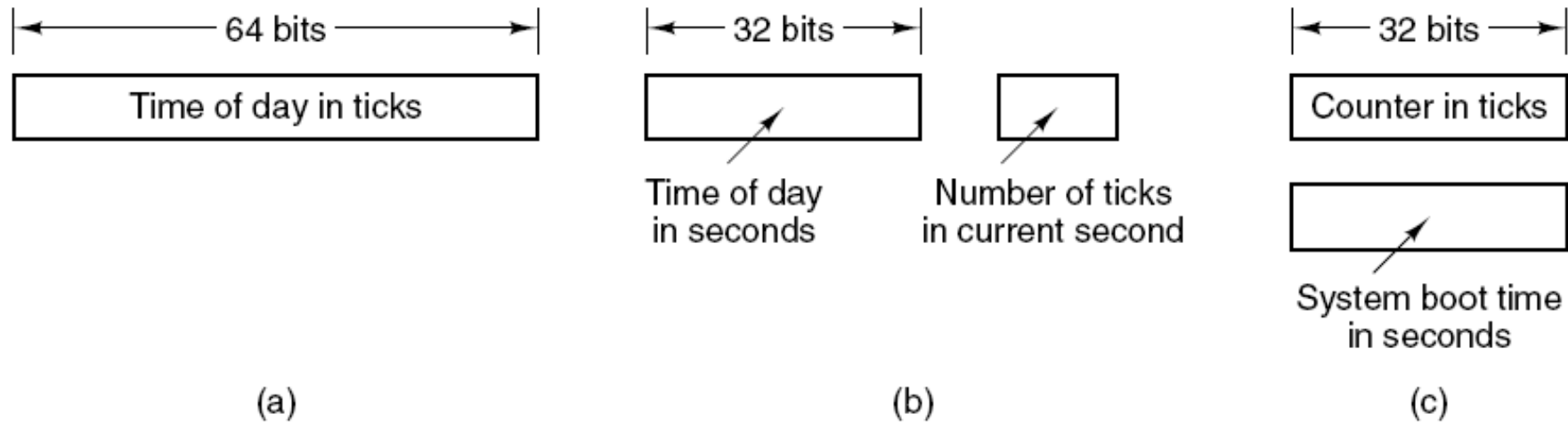
A programmable clock.

Clock Software (1)

Typical duties of a clock driver

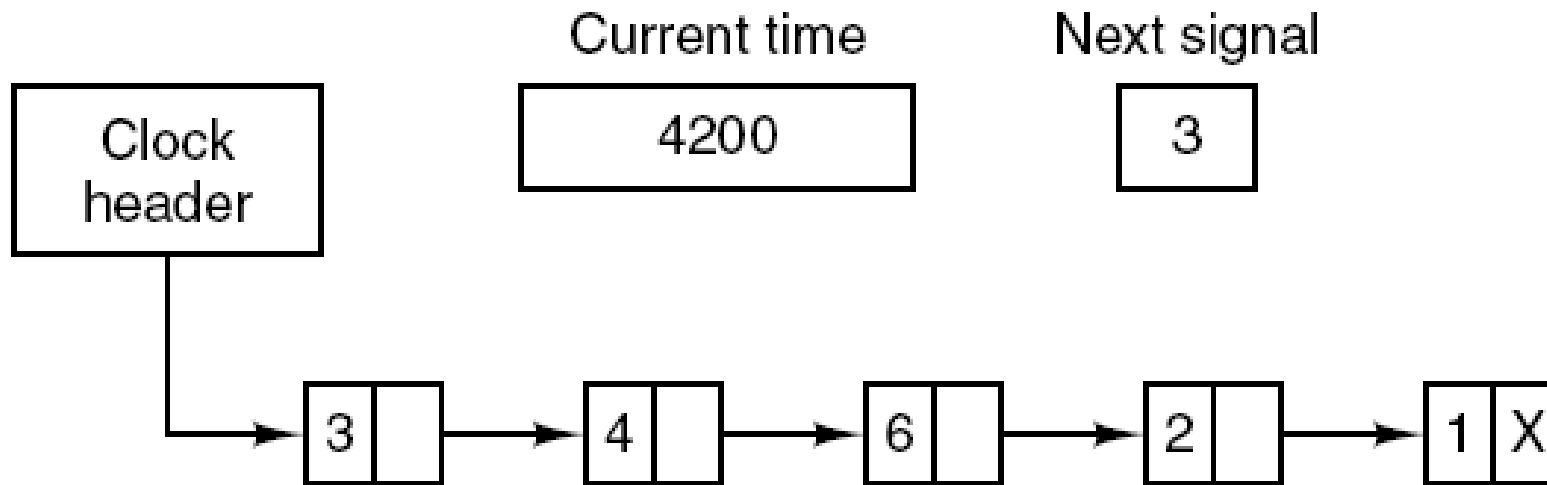
1. Maintaining the time of day.
2. Preventing processes from running longer than they are allowed to.
3. Accounting for CPU usage.
4. Handling alarm system call made by user processes.
5. Providing watchdog timers for parts of the system itself.
6. Doing profiling, monitoring, statistics gathering.

Clock Software (2)



Three ways to maintain the time of day

Clock Software (3)



Simulating multiple timers with a single clock.

Soft Timers

Soft timers succeed according to rate at which kernel entries are made because of:

1. System calls.
2. TLB misses.
3. Page faults.
4. I/O interrupts.
5. The CPU going idle.

Keyboard Software

Character	POSIX name	Comment
CTRL-H	ERASE	Backspace one character
CTRL-U	KILL	Erase entire line being typed
CTRL-V	LNEXT	Interpret next character literally
CTRL-S	STOP	Stop output
CTRL-Q	START	Start output
DEL	INTR	Interrupt process (SIGINT)
CTRL-\	QUIT	Force core dump (SIGQUIT)
CTRL-D	EOF	End of file
CTRL-M	CR	Carriage return (unchangeable)
CTRL-J	NL	Linefeed (unchangeable)

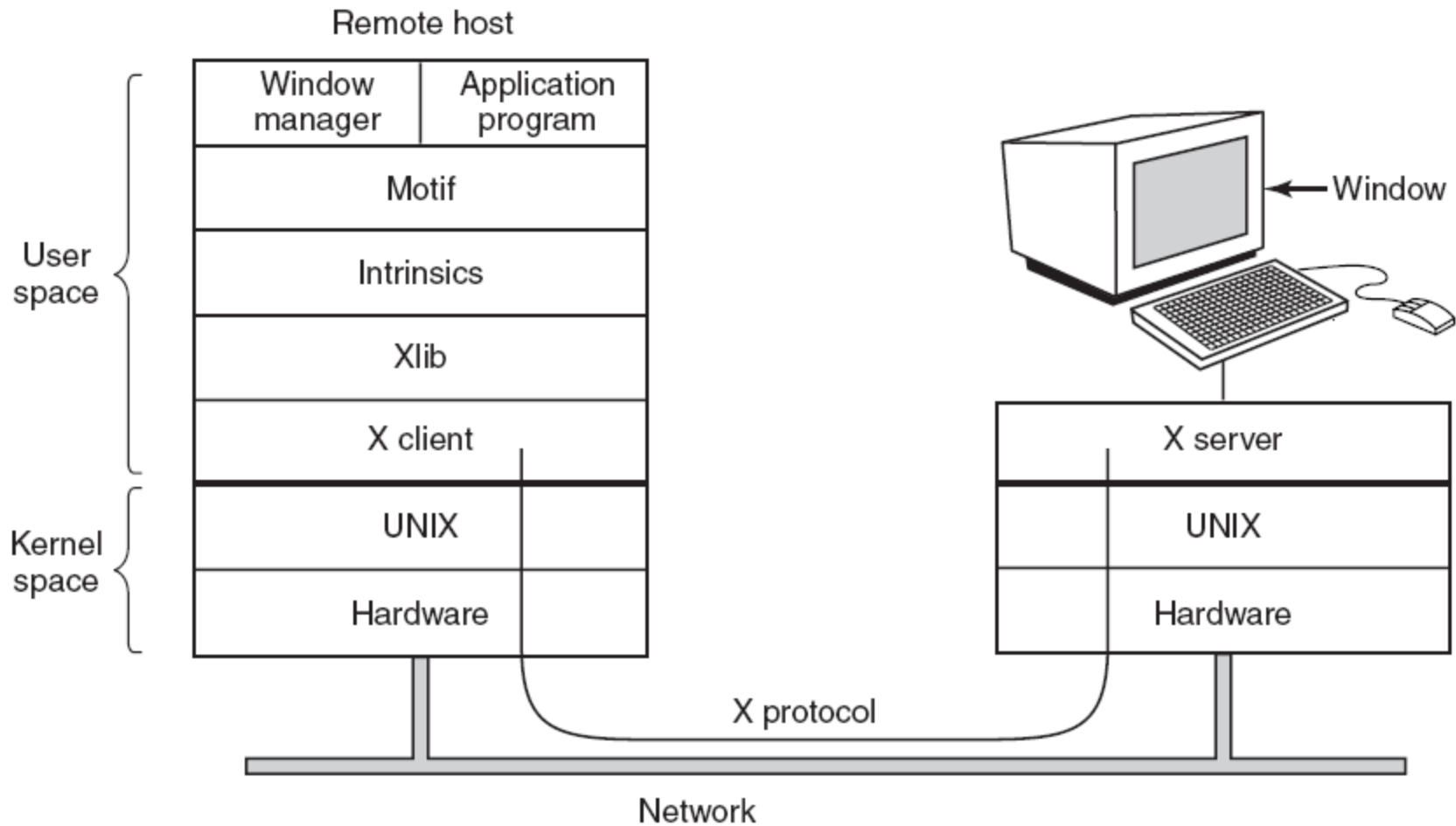
Characters that are handled specially in canonical mode.

The X Window System (1)

Escape sequence	Meaning
ESC [<i>n</i> A	Move up <i>n</i> lines
ESC [<i>n</i> B	Move down <i>n</i> lines
ESC [<i>n</i> C	Move right <i>n</i> spaces
ESC [<i>n</i> D	Move left <i>n</i> spaces
ESC [<i>m</i> ; <i>n</i> H	Move cursor to (<i>m</i> , <i>n</i>)
ESC [<i>s</i> J	Clear screen from cursor (0 to end, 1 from start, 2 all)
ESC [<i>s</i> K	Clear line from cursor (0 to end, 1 from start, 2 all)
ESC [<i>n</i> L	Insert <i>n</i> lines at cursor
ESC [<i>n</i> M	Delete <i>n</i> lines at cursor
ESC [<i>n</i> P	Delete <i>n</i> chars at cursor
ESC [<i>n</i> @	Insert <i>n</i> chars at cursor
ESC [<i>n</i> m	Enable rendition <i>n</i> (0=normal, 4=bold, 5=blinking, 7=reverse)
ESC M	Scroll the screen backward if the cursor is on the top line

The ANSI escape sequences accepted by the terminal driver on output. ESC denotes the ASCII escape character (0x1B), and *n*, *m*, and *s* are optional numeric parameters.

The X Window System (2)



Clients and servers in the M.I.T. X Window System.

The X Window System (3)

Types of messages between client and server:

1. Drawing commands from the program to the workstation.
2. Replies by the workstation to program queries.
3. Keyboard, mouse, and other event announcements.
4. Error messages.

Graphical User Interfaces (1)

```
#include <X11/Xlib.h>
#include <X11/Xutil.h>

main(int argc, char *argv[])
{
    Display disp;                /* server identifier */
    Window win;                  /* window identifier */
    GC gc;                       /* graphic context identifier */
    XEvent event;                /* storage for one event */
    int running = 1;

    disp = XOpenDisplay("display_name");    /* connect to the X server */
    win = XCreateSimpleWindow(disp, ... );    /* allocate memory for new window */
    XSetStandardProperties(disp, ...);        /* announces window to window mgr */
    gc = XCreateGC(disp, win, 0, 0);          /* create graphic context */
    XSelectInput(disp, win, ButtonPressMask | KeyPressMask | ExposureMask);
    XMapRaised(disp, win);                  /* display window; send Expose event */
}
```

Graphical User Interfaces (2)

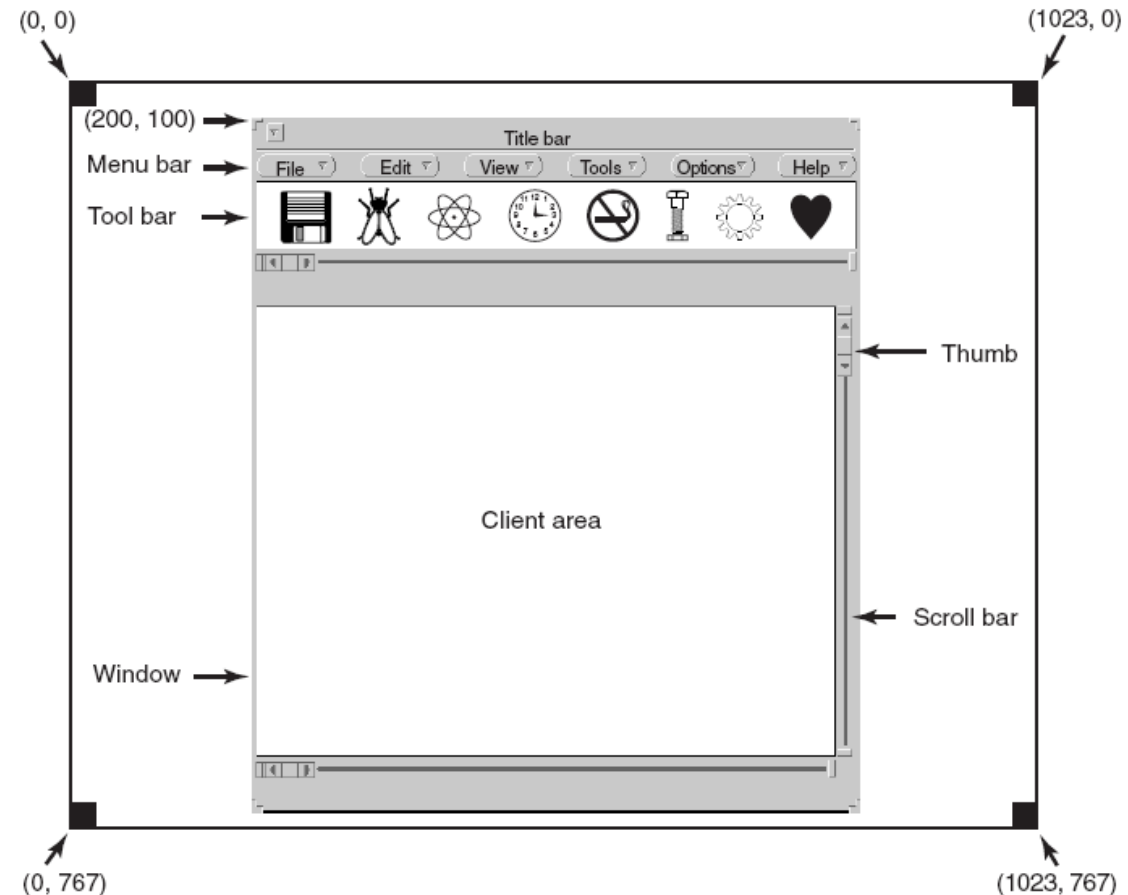
```
while (running) {
    XNextEvent(disp, &event);          /* get next event */
    switch (event.type) {
        case Expose:      ...; break;      /* repaint window */
        case ButtonPress: ...; break;      /* process mouse click */
        case Keypress:    ...; break;      /* process keyboard input */
    }
}

XFreeGC(disp, gc);                    /* release graphic context */
XDestroyWindow(disp, win);            /* deallocate window's memory space */
XCloseDisplay(disp);                  /* tear down network connection */
}
```

A skeleton of an X Window application program.

Graphical User Interfaces (3)

A sample window located at (200, 100) on an XGA display.



Graphical User Interfaces (4)

```
#include <windows.h>

int WINAPI WinMain(HINSTANCE h, HINSTANCE, hprev, char *szCmd, int iCmdShow)
{
    WNDCLASS wndclass;           /* class object for this window */
    MSG msg;                     /* incoming messages are stored here */
    HWND hwnd;                   /* handle (pointer) to the window object */

    /* Initialize wndclass */
    wndclass.lpfnWndProc = WndProc; /* tells which procedure to call */
    wndclass.lpszClassName = "Program name"; /* Text for title bar */
    wndclass.hIcon = LoadIcon(NULL, IDI_APPLICATION); /* load program icon */
    wndclass.hCursor = LoadCursor(NULL, IDC_ARROW); /* load mouse cursor */

    RegisterClass(&wndclass);      /* tell Windows about wndclass */
    hwnd = CreateWindow ( ... )    /* allocate storage for the window */
    ShowWindow(hwnd, iCmdShow);    /* display the window on the screen */
    UpdateWindow(hwnd);            /* tell the window to paint itself */
}
```

Graphical User Interfaces (5)

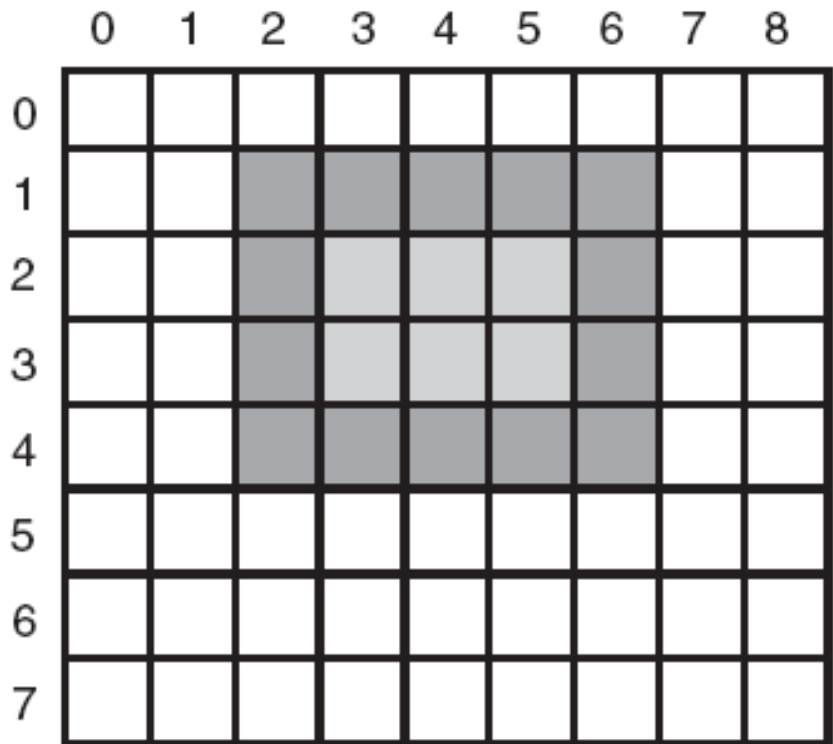
```
while (GetMessage(&msg, NULL, 0, 0)) {      /* get message from queue */
    TranslateMessage(&msg);                /* translate the message */
    DispatchMessage(&msg);                 /* send msg to the appropriate procedure */
}
return(msg.wParam);
}

long CALLBACK WndProc(HWND hwnd, UINT message, UINT wParam, long lParam)
{
    /* Declarations go here. */

    switch (message) {
        case WM_CREATE:    ... ; return ... ; /* create window */
        case WM_PAINT:     ... ; return ... ; /* repaint contents of window */
        case WM_DESTROY:   ... ; return ... ; /* destroy window */
    }
    return(DefWindowProc(hwnd, message, wParam, lParam)); /* default */
}
```

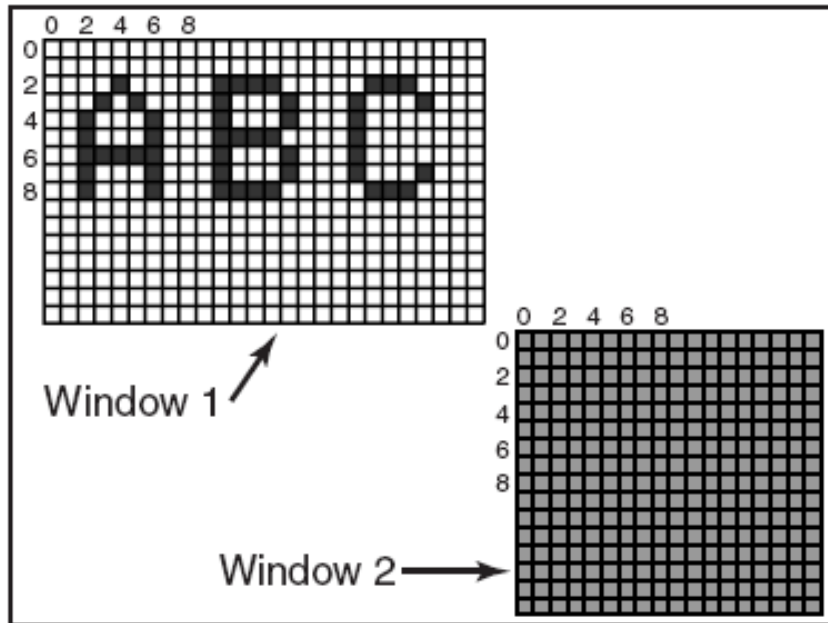
A skeleton of a Windows main program

Bitmaps (1)

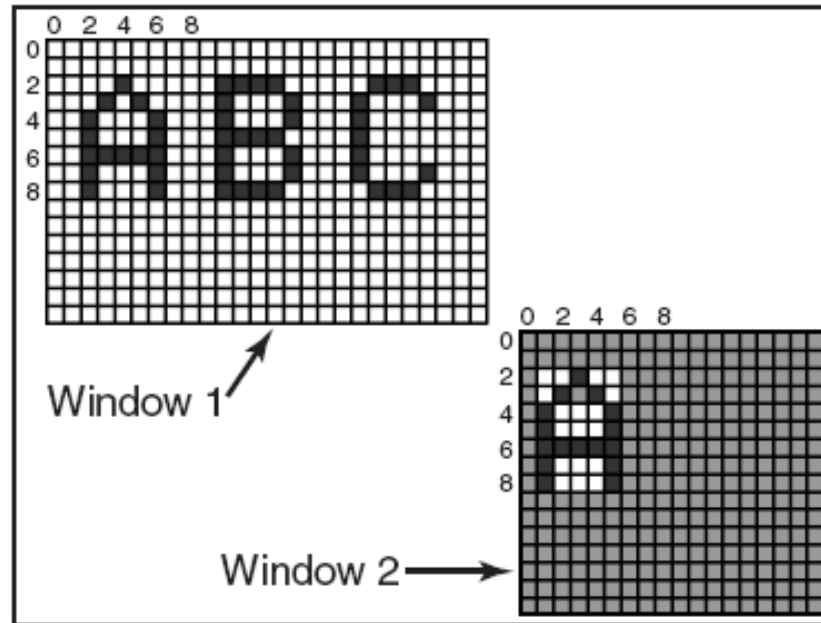


An example rectangle drawn using Rectangle.
Each box represents one pixel.

Bitmaps (2)



(a)



(b)

Copying bitmaps using *BitBlt*. (a) Before. (b) After.

Fonts

20 pt: abcdefgh

53 pt: abcdefgh

81 pt: abcdefgh

Some examples of character outlines at different point sizes.

Thin Clients

Command	Description
Raw	Display raw pixel data at a given location
Copy	Copy frame buffer area to specified coordinates
Sfill	Fill an area with a given pixel color value
Pfill	Fill an area with a given pixel pattern
Bitmap	Fill a region using a bitmap image

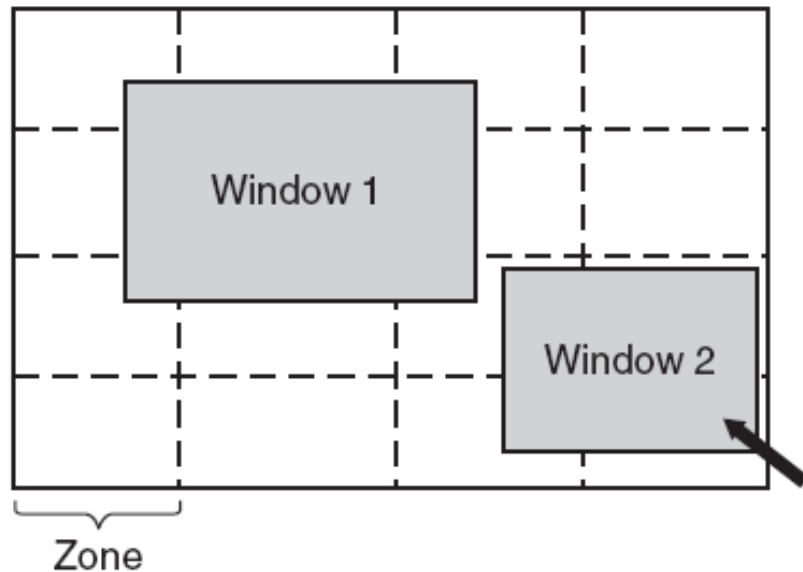
The THINC protocol display commands

Power Management Hardware Issues

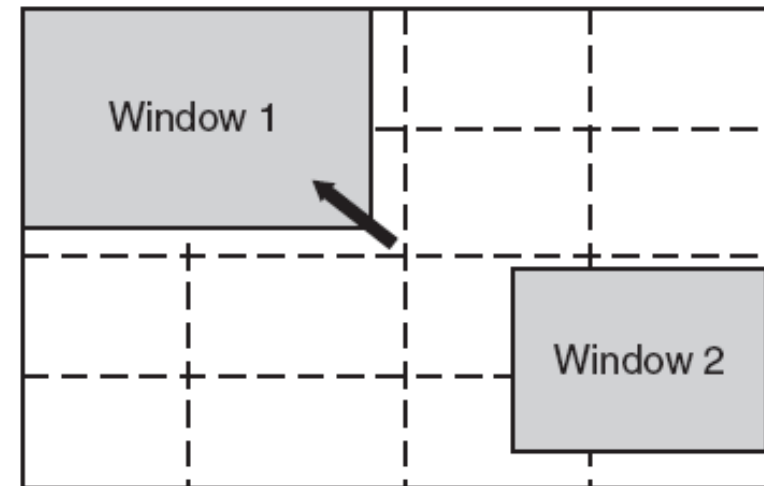
Device	Li et al. (1994)	Lorch and Smith (1998)
Display	68%	39%
CPU	12%	18%
Hard disk	20%	12%
Modem		6%
Sound		2%
Memory	0.5%	1%
Other		22%

Power consumption of various parts of a notebook computer

Power Management – The Display



(a)



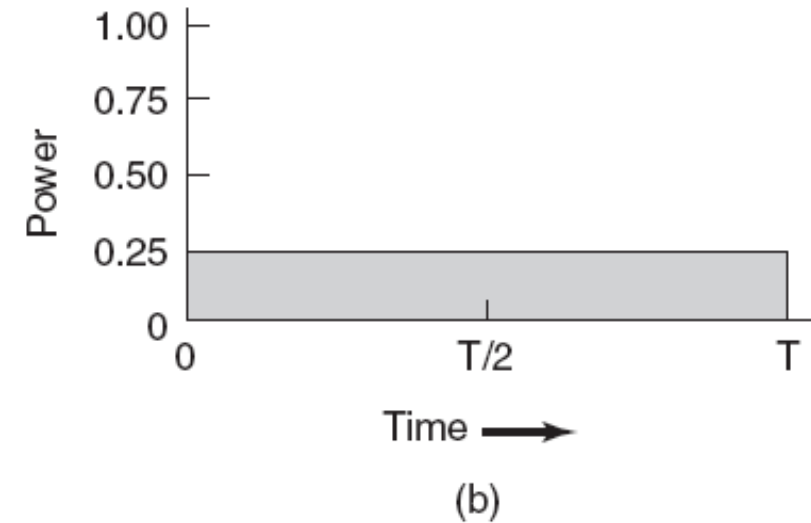
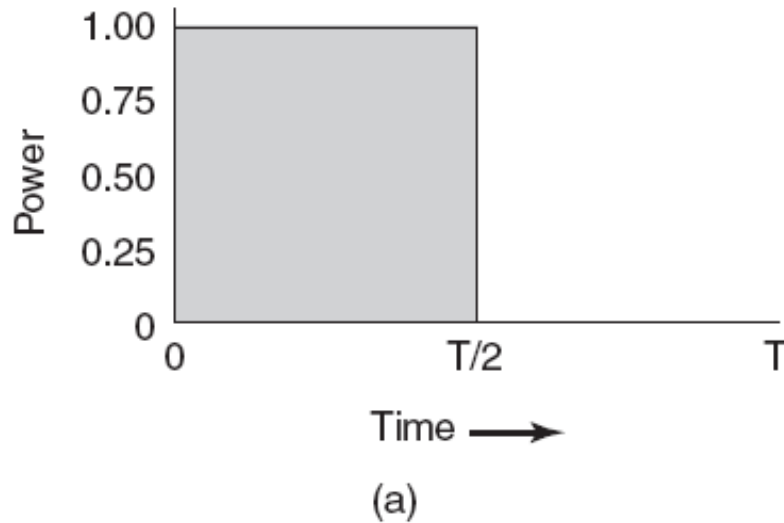
(b)

The use of zones for backlighting the display.

(a) When window 2 is selected it is not moved.

(b) When window 1 is selected, it moves to reduce the number of zones illuminated.

Power Management the CPU



(a) Running at full clock speed.

(b) Cutting voltage by two cuts clock speed by two and power consumption by four.

User-Level Daemons

- How is a replacement RAID drive reconstructed?
- Not everything is best done in the kernel
- It could be done in the kernel — but should it be?

What's in the Kernel?

- What are the criteria for putting something in the kernel
- On the other hand — why should we avoid putting things in the kernel?
- Different operating systems make different choices

In the Kernel

- Something that's in the kernel can't be changed by a user or a site
- Kernel bugs frequently crash the system
- There's no intra-kernel memory protection
- But — kernel-to-kernel operations are faster than kernel-to-user; there's no context switch involved
- For a user program to talk to a (privileged) user level program involves copying the data to and from the kernel; this can be expensive

Criteria

- Put something in the kernel if there should be only one way of doing it (i.e., loading the virtual memory registers)
- Put something in the kernel if it's needed before the system is really running (i.e., the disk I/O driver)
- Put something in the kernel if it needs frequent access to kernel-only resources (i.e., the page frame reclamation algorithm)
- Basically, put something in the kernel if and only if there's no choice

Avoiding the Kernel

- Put it in user space if possible
- If there's no significant performance issue, put it in userspace
- If there is a performance hit but it's rarely done, put it in user space
- If you may want to change its behavior, put it in userspace
- If you can stop and restart it, put it in userspace
- If it uses files for configuration, put it in userspace

Avoiding the Kernel

- Put it in user space if possible
- If there's no significant performance issue, put it in userspace
- If there is a performance hit but it's rarely done, put it in user space
- If you may want to change its behavior, put it in userspace
- If you can stop and restart it, put it in userspace
- If it uses files for configuration, put it in userspace

Kernel Interfaces

- If we're going to put things in userland, we need proper interfaces
- Linux relies on pseudo-file systems — /proc, /sys, and more — for many of its interfaces
- Not a bad idea — everything appears to be a file, and file permissions can be used to restrict access

Daemons

- Daemon: “A program that runs unattended to perform continuous or periodic systemwide functions”
- Name originally used in Multics; some say it came from Maxwell’s Demon, but multicians.org says it’s from the Greek word $\mu\iota$: “A supernatural being of a nature intermediate between that of gods and men”
- Either way — daemons perform system functions, but are not part of the operating system

Example: The Line Printer Daemon

- We can't let user programs talk directly to the printer
- Instead, we let them talk to the printer daemon; it talks to the printer

Why Can't User Programs Talk to the Printer?

- Speed — the printer is slower than programs
- Conflict — more than one program may need to print simultaneously
- Accounting — many sites charge users for pages printed
- Security — prevent interference with another job
- Labeling — make sure output is properly labeled
- Indirection — multiple printers, different output formats, etc

The Line Printer Daemon

- Several pieces — user-facing, printer-facing, administrative
- User-facing: invoked by user with files or output to print
- Printer-facing: picks up requests submitted by users; copies them to printer
- Administrative: group printers, set permissions, set up or down, describe output format, etc.

Communications

- How do the different pieces talk?
- Pipes? Files? Signals? Network connections? Something else?
- Any and all of the above!
- But — must be careful about security?

Security and the Printer Daemon

- Users must not be able to overwrite printer daemon files
- Easy — use ordinary Unix file permissions
- Make user-facing program setuid
- Files to be printed must be readable by the user (for security) and by the printer daemon (for efficiency)
- If not readable by the daemon, copy the file; if not readable by the user, scream

Linking versus Copying

- It's tempting to create a link to the file the user wants printed
- Very difficult — does the user have read permission on it?
- Watch out for race conditions

How to Link

- With user's permissions, determine the current directory and rewrite relative paths as absolute paths
- With daemon's permissions, change to a world-writable directory underneath a protected directory
- With user's permissions, create a hard link to the file in this world-writable directory
- With user's permissions, verify that it can be opened
- Don't record any decisions on size, formatting, etc., now, because the user still has a link and ownership, and can change that stuff

Special Code and Pseudo-Devices

- On Unix, the printer daemon is easy because the printer, too, is just a file.
- Programs that expect to talk to the printer directly can generally talk to the daemon via a pipe, or write to a file, with no changes
- On other operating systems, applications need special printer interface code
- This can be good, because it lets the programs know what resolution output to generate
- On some systems, pseudo-devices — software that looks like a hardware printer — must be used

Daemons and Permissions

- What permissions do daemons run with?
- Some run as root, especially those that need to permit logins (i.e., some network daemons)
- Better to use dedicated login: lpd, games, named, etc.
- On Multics, most daemons ran in group sysdaemon or netdaemon
- Principle of Least Privilege

NFS

- NFS (Network File System) uses user- and kernel-mode pieces
- The actual disk I/O is in the kernel, for performance reasons
- But file locking is done in a user-level daemon. Why?

NFS Lock Daemon

- NFS, by design, is stateless; clients and servers can be rebooted freely
- Also helps guard against network outages
- But locking is inherently stateful
- Lock daemon keeps track of file locks; stores them on lock disk, for crash recovery

Other Vital Commands

- Login and X
- Mount — make file systems available
- Fsck — repair file systems after a crash
- Halt and reboot
- Shells
- Init

Login

- Runs as root, but is not setuid; always invoked as root
- Accepts login name and password
- Sets up user environment

The X Window System

- On Unix, X Windows is not a part of the kernel
- The only thing special about X is that it has permission to open the graphics card
- Admittedly, that often requires root privileges
- Frequently has a login component: xdm, gdm, etc.

Shells

- No required user shell
- Example: CS dept machines have at least seven different real shells in use
- Can have special-purpose shells for restricted logins
- But — system administration and startup require `/bin/sh`
- For (complex) security reasons, sometimes need a list of “official” general-purpose shells

/sbin/init

- Process 1 — the beginning and end of all processes
- Started by the kernel at boot time — a hand-crafted process table entry
- Initiates system setup

System Setup

- init reads a configuration file describing run levels and what to do when entering them
- One pseudo-run level is sysinit: system initialization
- Sysinit: sets host names, mounts /proc and /sys, initializes certain hardware, runs fsck

Entering Run Levels

- Different run-levels select different system services
- “1” is single-user, “2” is multi-user without NFS, “3” is everything, “5” is X11, “6” is reboot
- init invokes `/etc/rc` to change run levels; it invokes all files in `/etc/rcN.d` to enter run level N

/etc/rcN.d

- Contains K and S scripts
- S starts a service; K kills one
- Executed in numerical order
- Examples: K50nfs, S80sendmail

See you next time!